

Design and Implementation of the APB Protocol on FPGA

**A REPORT
ON
PROJECT OF
INDUSTRY DEFINED PROJECT (4EL33)**

*Bachelor of Technology
(Electronics Engineering)*

Submitted by:

**ROHIT VIJAY GUPTA
(21EL044)**

Under the guidance of:

Dr. Tanmay D. Pawar

Industry Name:

Einfochips, an Arrow company

Industry Guide:

Mr. Ashishkumar Christian

Ms. Matri Patel



Birla Vishvakarma Mahavidyalaya
Engineering College (An Autonomous Institution)
Vallabh Vidyanagar - 388 120
Affiliated to Gujarat Technological University

Certificate

This is to certify that the Project-I entitled “**Design and Implementation of the APB Protocol on FPGA**” has been carried out by **Rohit Gupta** in the **8th semester** under my guidance in Electronics Engineering at **Birla Vishvakarma Mahavidyalaya**, Vallabh Vidyanagar, during the academic year 2024–25.

Dr. Tanmay D Pawar

Head of the Department

Dept. of Electronics Engineering

Birla Vishvakarma Mahavidyalaya

Dr. Deepak L Vala

Course Co-ordinator

Dept. of Electronics Engineering

Birla Vishvakarma Mahavidyalaya

Ashishkumar Christian

Industry Guide

Senior Engineer (Level 2)

Einfochips, an Arrow company

Ahmedabad

Maitri Patel

Industry Guide

Senior Engineer (Level 1)

Einfochips, an Arrow company

Ahmedabad



Provisional Training Certificate

From DIPESH PANCHAL <DIPESH.PANCHAL@einfochips.com>

Date Tue 4/29/2025 12:45 PM

To Rohit Gupta <Rohit.Gupta@einfochips.com>

Cc Maitri Patel <Maitri.Patel@einfochips.com>

This is to certify that Mr. Rohit V. Gupta (ID : 21EL044) in B.Tech. Semester VIII (Electronics Engineering) of Birla Vishvakarma Mahavidyalaya Engineering College , Anand is provided training at eInfochips, Ahmedabad from 06-01-2025 to 30-04-2025.

He has satisfactorily completed the task assigned to him to fulfill the academic requirements.

Regards.

Dipesh Panchal

ASIC Engineer

eInfochips, An Arrow Company

M:9879942991



Acknowledgement

It is with immense pride and sincere gratitude that we present this final year project entitled “**Design and Implementation of the APB Protocol on FPGA.**” This project marks a significant milestone in our academic journey, reflecting both the theoretical knowledge gained over the course of our engineering studies and its practical application in the domain of digital system design and embedded communication protocols.

We express our deepest gratitude to **Dr. Tanmay Pawar**, our project guide and Head of the Department, Electronics Engineering, Birla Vishvakarma Mahavidyalaya, Vallabh Vidyanagar, Anand. His invaluable mentorship, constant encouragement, and insightful technical guidance have been the cornerstone of this project. His enthusiasm for innovation and clarity of thought have greatly influenced our approach and understanding throughout the course of this work.

We are also profoundly thankful to **Birla Vishvakarma Mahavidyalaya (BVM)** for providing us with an excellent academic environment, modern laboratory facilities, and the infrastructure necessary to successfully carry out this project. The support from the department, both in terms of technical resources and moral encouragement, played a vital role in our development as engineers.

We are especially grateful to **Einfochips, an Arrow Company**, for giving us the opportunity to gain industry exposure through this project. The collaboration with Einfochips has been a highly enriching experience, allowing us to align our academic knowledge with real-world industry practices. We extend our sincere thanks to our industry mentors, **Mr. Ashishkumar Christian** and **Ms. Matri Patel**, for their expert guidance, timely feedback, and continuous support. Their professional insights and practical perspectives added immense value to the execution and implementation of this project.

Their collective support, from both academic and industry mentors, has significantly contributed to the depth and quality of our work. This project stands as a testament to the spirit of collaboration between academia and industry, and we are truly grateful for this unique learning opportunity.

This project is not just the result of technical implementation, but also a journey of growth, collaboration, and learning. We sincerely hope that our work makes a meaningful contribution to the field of digital design and serves as a foundation for further exploration in FPGA-based protocol implementations.

Abstract

This project presents the **design and implementation of the Advanced Peripheral Bus (APB) protocol** on an FPGA platform, using the **Artix-7 Nexys 4 development board**. The APB protocol is a key component of the **AMBA (Advanced Microcontroller Bus Architecture)** family, developed by ARM, and is known for its simplicity, low power consumption, and suitability for connecting **low-bandwidth peripherals**. The work begins with a comprehensive understanding of the APB protocol and its signal-level behavior. A **master-slave architecture** is developed where a single APB master communicates with multiple APB-compatible slave modules through a set of well-defined control signals and timing phases.

To validate the correctness of the design, a series of **simulation test scenarios** were developed and executed using **Vivado**. These include a **Sanity Test** for basic read and write functionality, a **Back-to-Back Write and Read** test to examine continuous transactions, a **Burst (5x) Write and Read** sequence to simulate high-speed data access, an **Asynchronous Reset Between Operations** test to analyze system behavior under reset conditions, and an **Invalid Address Access** test to verify handling of undefined address spaces. The **simulation waveforms** obtained from Vivado confirmed the accurate implementation of protocol signals and handshake mechanisms, validating the timing and control logic of the APB interface.

Following the successful simulation, the APB design was extended by implementing three custom **APB slave modules**: a **RAM (memory)** unit for data storage, a **7-segment display controller** for numeric output display, and a **UART** interface for serial data communication. These slaves were connected to the APB master through a shared bus and were individually verified in both simulation and hardware. The **design hierarchy and signal mapping** were carefully organized to ensure correct slave selection and data routing during APB transactions.

The final system was synthesized and deployed on the **Artix-7 Nexys 4 FPGA board**, and its functionality was verified in real-time. The **UART module** successfully communicated with a PC terminal, the **7-segment display** responded accurately to data commands, and the **RAM** provided reliable storage and retrieval of values. This project not only confirms the successful implementation of the APB protocol but also showcases the integration of practical peripheral devices on FPGA hardware. The work highlights the use of **Vivado for simulation and synthesis**, and demonstrates a complete flow from design to verification and deployment, offering valuable insights for **embedded system design** and **SoC integration**.

Keywords: APB protocol, AMBA, FPGA, Artix-7, Nexys 4, UART, RAM, 7-segment display, Vivado, master-slave design, hardware verification, embedded systems, bus interface.

Table of Contents

Certificate	ii
Industry Certificate.....	iii
Acknowledgement	iv
Abstract	v
List of Figures	viii
List of Table	ix
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Motivation	2
1.3 Objectives	2
1.4 Thesis Organization	3
Chapter 2: Literature Review	5
Chapter 3: APB Protocol Fundamentals	7
3.1 APB Protocol Features	7
3.2 APB Protocol Signals	7
3.3 Detailed Explanation of Each Signal.....	8
3.3.1 PCLK (Clock).....	8
3.3.2 PRESETn (Active-low Reset)	9
3.3.3 PADDR (Address Bus).....	9
3.3.4 PSEL (Peripheral Select).....	10
3.3.5 PWRITE (Write Control Signal)	10
3.3.6 PWDATA (Write Data Bus)	10
3.3.7 PRDATA (Read Data Bus).....	11
3.3.8 PENABLE (Enable Signal)	11
3.3.9 PREADY (Ready Signal)	11
3.3.10 PSLVERR (Slave Error Signal)	12
3.4 APB Timing Diagrams with Wait States and Error Handling.....	12
3.4.1 Write Operation Without Wait States	12
3.4.2 Write Operation With Wait States.....	13

3.4.3 Read Operation Without Wait States.....	13
3.4.4 Read Operation With Wait States	14
3.4.5 Error Handling (PSLVERR)	15
Chapter 4: Reference Architecture of APB Protocol	16
4.1 Overview of APB Architecture	16
4.2 Master-Slave Communication Flow	16
4.3 Timing and Control Signals in Reference Design.....	18
4.3.1 APB Signal Timing Behavior.....	19
4.4 APB Bus Interconnect Structure.....	20
Chapter 5: RTL Design and Simulation	22
5.1 Design Flow	22
5.2 RTL Coding of APB Modules	23
5.3 Testbench Development	24
5.4 Simulation Results	24
Chapter 6: Slave Devices Design and Selection	27
6.1 Slave Selection	27
6.2 Memory Slave.....	28
6.2.1 Objective	28
6.2.2 Design Overview	28
6.2.3 Functional Description	29
6.2.4 Verilog HDL Implementation Overview.....	30
6.2.5 APB Signal Timing Behavior.....	30
6.2.6 Simulation and Testing.....	30
6.3 7-Segment Display.....	31
6.3.1 Working Principle	31
6.3.2 Hexadecimal Values Representation	32
6.3.3 Connecting a 7-Segment Display to APB	32
6.4 UART	33
6.4.1 How UART Works.....	33
6.4.2 Frame Format	34
6.4.3 Start Bit.....	34
6.4.4 Data bit	34

6.4.5 Parity bit.....	35
6.4.6 Stop bit	35
6.4.7 UART Transmitter Module (Summary).....	36
6.4.8 UART Receiver Module (Summary).....	36
Chapter 7: FPGA Implementation	37
7.1 FPGA Board Description: Artix-7 Nexys 4	37
7.1.1 Overview of Nexys 4	37
7.1.2 Key Features of Nexys 4 Board	38
7.1.3 Relevance to the APB Protocol Design	38
7.1.4 Vivado Toolchain Compatibility.....	39
7.2 FPGA Pin Configuration and Component Mapping	39
7.2.1 Clock Signal	40
7.2.2 User Inputs	40
7.2.3 Output Indicators	41
7.2.4 UART Interface	41
7.3 Hardware Setup	41
7.4 Memory Operations	42
7.4.1 Write Operation.....	42
7.4.2 Read Operation	42
7.5 7-Segment Display Operations	43
7.5.1 Write Operation.....	43
7.5.2 Read Operation	43
7.6 UART Operations.....	43
7.6.1 Write Operation (Transmit Data).....	43
7.6.2 Read Operation (Receive Data)	43
Chapter 8: Results and Discussion	44
8.1 RTL Schematic Overview.....	44
8.2 Power Analysis Report	45
8.3 Utilization Report.....	45
8.4 Discussion	46
Chapter 9: Applications and Use Cases.....	47
9.1 Real-World Applications	47
9.2 Scope for Further Enhancement	47

Chapter 10: Conclusion	48
10.1 Summary of the Project.....	48
10.2 Key Achievements	48
10.3 Contributions to the Field.....	49
10.4 Limitations and Challenges.....	49
10.5 Final Thoughts.....	50
References.....	51
Appendix	52
Chapter A : APB Protocol Verilog Implementation.....	52
APB Master Verilog Code.....	52
APB Slave Verilog Code	54
APB Top-Level Integration Module.....	56
APB Testbench.....	58

List of Figures

Figure 3.1:	Write Operation Without Wait States	13
Figure 3.2:	Write Operation With Wait States	13
Figure 3.3:	Read Operation Without Wait States	14
Figure 3.4:	Read Operation With Wait States	14
Figure 3.5:	Write Operation with Error	15
Figure 4.1:	Reference Architecture of APB Protocol	17
Figure 4.2:	State Diagram of APB Protocol	19
Figure 4.3:	APB Interconnect architecture	21
Figure 5.1:	Waveform for Sanity Test (Write and Read Same Address)	24
Figure 5.2:	Waveform for Back-to-Back Write and Read	25
Figure 5.3:	Waveform for 5x Write and Read Burst	25
Figure 5.4:	Waveform Showing Asynchronous Reset in Middle of Transaction	25
Figure 5.5:	Waveform Showing Slave Error on Invalid Address Access	26
Figure 6.1:	APB Master with Three Slaves (UART, 7-Segment Display, Memory)	27
Figure 6.2:	anode configuraution	31
Figure 6.3:	UART Serial Communication	34
Figure 6.4:	UART Frame Format	34
Figure 6.5:	UART Start bit	34
Figure 6.6:	UART Data bit	35
Figure 6.7:	UART Parity bit	35
Figure 6.8:	UART Stop bit	35
Figure 7.1:	Artix-7 Nexys 4 FPGA Board	37
Figure 7.2:	Labeled Nexys 4 FPGA board showing APB signal mapping	40
Figure 7.3:	Hardware Setup	42
Figure 8.1:	RTL schematic of APB Protocol	44
Figure 8.2:	Power analysis report of the implemented APB Protocol	45
Figure 8.3:	Vivado utilization report of the APB Protocol	46

List of Tables

Table 3.1:	APB Protocol Signals	8
Table 4.1:	APB Signal Timing Behavior	20
Table 6.1:	Slave Selection.....	28
Table 6.2:	APB Signal Timing Behavior	30
Table 6.3:	Hexadecimal values and their corresponding segments.....	32
Table 8.1:	FPGA Resource Utilization Summary	46

Chapter 1: Introduction

1.1 Background

Embedded systems have become an integral part of modern technology, with applications ranging from consumer electronics to automotive systems and medical devices. As the complexity of embedded systems increases, so does the need for efficient communication protocols that allow different components to interact seamlessly. The design of these communication interfaces plays a crucial role in determining the overall performance, power consumption, and scalability of the system.

In the context of System-on-Chip (SoC) designs, various communication protocols are employed to ensure that the processing unit can efficiently interact with peripheral devices such as memory, UARTs, timers, sensors, and more. One such protocol that is widely adopted in the semiconductor industry is the **Advanced Microcontroller Bus Architecture (AMBA)**, developed by ARM. The AMBA family of protocols includes the **Advanced eXtensible Interface (AXI)**, **Advanced High-performance Bus (AHB)**, and the **Advanced Peripheral Bus (APB)**.

Of these, the APB protocol stands out due to its simplicity and low overhead, which make it ideal for connecting low-bandwidth peripherals like UARTs, timers, and GPIOs to the processing unit. The APB protocol is designed to provide a lightweight interface for low-speed, low-power peripherals in a way that doesn't burden the system's resources. It simplifies communication by minimizing the control signals required for peripheral interfacing, making it ideal for power-sensitive applications.

The motivation for this project lies in providing a real-world implementation of the APB protocol, allowing engineers and students to bridge the gap between theoretical knowledge and hands-on hardware experience. By implementing the APB protocol on an FPGA platform (Artix-7 on Nexys 4 board), this project provides insights into the practical challenges involved in creating a communication interface, such as signal timing, synchronization, and real-time data handling.

Moreover, this project emphasizes the importance of modularity and scalability in embedded system design. The APB protocol, being simple and well-structured, supports easy integration of peripherals, making it an excellent choice for designing scalable systems. The implementation of APB will also lay the foundation for exploring more complex protocols like AXI and AHB in future projects.

1.2 Motivation

The embedded systems field is driven by the need for efficient communication between processing units and peripheral devices. Historically, custom communication interfaces have been used to connect various peripherals, but these are often time-consuming to design and difficult to scale. Furthermore, they lack standardization, making it challenging to integrate multiple devices or port designs to different platforms.

As SoC designs become more complex and integrated, there is an increasing demand for standardized communication protocols that can handle diverse peripherals and devices. The APB protocol provides a solution to this problem by offering a simple, low-overhead interface for connecting peripheral devices in an SoC. It is particularly suited for low-speed, low-power peripherals that don't require the high-bandwidth or complex signaling offered by protocols like AXI or AHB.

In the context of this project, the motivation is to demonstrate the practical implementation of the APB protocol in a real-world hardware environment. By choosing to implement the protocol on an FPGA, the project provides valuable hands-on experience in designing and testing hardware interfaces. FPGA platforms, such as the Nexys 4 with Artix-7, offer the flexibility to design, simulate, and test communication protocols in a way that would be difficult to achieve with traditional microcontroller-based systems.

The project also aims to address the educational gap where students often study protocol theory without having the opportunity to implement and test them in real hardware systems. This implementation will provide insights into the challenges faced during real-time communication, signal integrity, and timing synchronization. Additionally, it will emphasize the importance of modularity and scalability, making it easier to expand the system in the future by adding new peripherals.

1.3 Objectives

The primary objectives of this project are to design, implement, and validate the AMBA APB protocol in a hardware environment. The specific objectives are as follows:

- **Design the APB protocol:** The first step is to design the APB protocol using Verilog HDL. This includes creating the APB master and slave modules, as well as defining the protocol signals (e.g., PSEL, PWRITE, PENABLE, PREADY, etc.). The design will follow the official AMBA specifications to ensure compatibility and correctness.

- **Develop an APB Master and Slave Modules:** The APB master will initiate communication and send requests to the APB slaves, which may include peripherals such as UART, memory, or a 7-segment display. The design will include multiple slaves to show how the APB protocol can interface with different peripheral devices. The slave modules will be designed to receive and process requests from the master.
- **Simulate and Verify the System:** Simulation is a crucial step in verifying the functionality of the design. Using Xilinx Vivado, the entire system will be simulated to check signal-level behavior and ensure proper communication between the master and slave modules. Waveform analysis will be conducted to verify the timing and synchronization of the signals.
- **Deploy the Design on FPGA:** After simulation, the design will be synthesized and deployed on the Nexys 4 FPGA board. Real-time testing will be conducted using physical switches, buttons, and output devices (e.g., LEDs, displays) to validate the APB communication and ensure that the system operates as expected in hardware.
- **Evaluate Synchronization and Functional Integrity:** The key objective of the project is to ensure that communication between the APB master and slave modules is synchronized. The project will evaluate read and write operations to confirm the integrity of data transfers. Any issues such as delays or misalignments will be documented.
- **Demonstrate Modularity and Scalability:** One of the strengths of the APB protocol is its modularity. The design will demonstrate how easily new peripherals can be added to the system. This will allow future expansion of the system with minimal effort.
- **Document and Analyze Performance:** Finally, the project will analyze the system's performance in terms of timing, area, and resource utilization. This will provide valuable insights into the efficiency of the APB protocol and highlight any potential bottlenecks or optimization opportunities.

1.4 Thesis Organization

This thesis is organized into ten chapters, each focusing on a specific stage of the project:

- **Chapter 1: Introduction**

Provides the background, motivation, and objectives of the project along with an overview of the thesis structure.

- **Chapter 2: Literature Review**
Reviews previous work on AMBA protocols and FPGA-based system designs, highlighting the need for implementing the APB protocol.
- **Chapter 3: APB Protocol Fundamentals**
Explains the APB protocol, including its signal definitions, timing diagrams, and operation sequences.
- **Chapter 4: Reference Architecture of APB Protocol**
Describes the system architecture involving the APB master and slave modules and their interaction.
- **Chapter 5: RTL Design and Simulation**
Covers the Verilog-based RTL design, testbench development, and simulation results using Xilinx Vivado.
- **Chapter 6: Slave Devices Design and Selection**
Discusses the implementation and role of the three APB-compliant slave modules: UART, 7-segment display, and memory.
- **Chapter 7: FPGA Implementation**
Details the synthesis and hardware implementation of the system on the Artix-7 Nexys 4 FPGA board.
- **Chapter 8: Results and Discussion**
Presents the simulation and hardware results, along with analysis and performance evaluation.
- **Chapter 9: Applications and Use Cases**
Outlines the practical applications of the APB protocol in embedded systems and SoC designs.
- **Chapter 10: Conclusion**
Summarizes the project, key outcomes, and suggests future improvements.

Chapter 2: Literature Review

The ARM AMBA (Advanced Microcontroller Bus Architecture) protocol suite plays a crucial role in the development of embedded and SoC designs. Among the protocols under the AMBA umbrella, the Advanced Peripheral Bus (APB) is optimized for connecting low-bandwidth peripherals. Rai and Yadav [1] present a comprehensive comparison between APB, AHB, and AXI protocols, highlighting that while AHB and AXI support high-speed pipelined transactions, APB is structured for simple, low-power peripheral communication. Their paper emphasizes that despite its simplicity, the APB protocol is integral to ensuring efficient processor-peripheral communication, especially when verified using SystemVerilog and UVM.

Further analysis of APB's adaptability in resource-constrained environments was presented by Neeharika et al. [2], who demonstrated that APB can be effectively implemented using open-source tools like Icarus Verilog and GTKWave. Their work provides an end-to-end APB design flow and highlights the protocol's deterministic nature, making it easier to verify and simulate compared to more complex protocols. They emphasize its value in cost-sensitive or academic SoC developments where commercial EDA tools may not be accessible.

Jain and Rao [3] contributed significantly by detailing the internal mechanics and timing structure of APB transactions. They describe the control FSM and its transitions through IDLE, SETUP, and ENABLE states. Their study includes comprehensive waveform analyses and coverage of corner cases, enabling a precise understanding of APB's signal behavior during read and write cycles. Their work validates how the use of synchronization signals such as PREADY ensures accurate handshaking between masters and slaves.

Practical design and synthesis concerns were addressed by Rayasa et al. [4], who proposed a clean, synthesizable Verilog implementation of the APB protocol. Their design uses an FSM-driven approach that minimizes resource usage while maintaining functional accuracy. They verified their implementation through simulation and also provided synthesis results demonstrating that the APB logic can be efficiently mapped onto FPGA hardware with minimal area overhead, validating its suitability for real-time embedded systems.

The broader perspective of APB's integration into SoC was presented by Yadav et al. [5], who elaborated on the complete master-slave interface, address decoding mechanism, and slave select logic. Their FPGA-based implementation was conducted using the Xilinx toolchain, where the APB bus was effectively synthesized and tested. They emphasized the importance of well-defined timing control in ensuring reliable inter-module communication on the FPGA fabric, particularly when multiple APB slaves are used.

Bolla et al. [6] explored the advantages of SystemVerilog in the RTL design of the APB protocol. Their work showcases how language features such as enumerations, interfaces, and typedef enable better abstraction and code maintainability. They also introduced SystemVerilog assertions to detect protocol violations during simulation, increasing design robustness. These features are essential for scalable and modular APB implementations in professional-grade hardware design.

A unique aspect of APB design arises when bridging it with other protocols such as AHB. Shanthi et al. [7] designed an AHB-to-APB bridge and successfully implemented it on an FPGA platform. Their study highlights signal synchronization, clock domain crossing, and control signal mapping as key challenges in such bridge architectures. The authors reported successful synthesis results and real-time performance benchmarks, showcasing the practicality of APB even when interfaced with higher-bandwidth protocols.

The verification aspect of APB was extensively covered by HK and Kumar [8], who created a structured UVM-based test environment for APB v2.0. Their verification suite covered a variety of test cases, including back-to-back writes, concurrent accesses, and error conditions. Functional coverage metrics and assertions were used to ensure complete FSM transition validation and correct data propagation. This methodology strengthens the reliability of APB implementations in production silicon.

Liu et al. [9] expanded on this by verifying an AXI-to-APB bridge system. Their work discusses the layered verification architecture, focusing on checking timing dependencies, transaction conversions, and AXI burst to APB single transfer translation. They suggest that as SoCs grow more complex, the combination of AXI for high-performance modules and APB for control-centric peripherals remains an ideal architectural choice. The study also outlines future scalability considerations, including low-latency bridge designs and dynamic slave mapping.

Chapter 3: APB Protocol Fundamentals

The Advanced Peripheral Bus (APB) is a communication protocol used in ARM's AMBA (Advanced Microcontroller Bus Architecture), designed specifically to connect low-bandwidth peripherals to the core system. APB is a simple and efficient protocol that is ideal for use in embedded systems, offering low power consumption and low complexity while still supporting essential data communication operations.

3.1 APB Protocol Features

The APB protocol offers the following key features:

- **Low Power:** Optimized for low-power consumption, APB is particularly suitable for peripherals that do not require high bandwidth.
- **Simple:** With fewer signals and a straightforward protocol, it is easier to implement and debug compared to higher-performance buses like AHB or AXI.
- **Non-Pipelined:** Data transfers in APB are not pipelined, meaning each transaction completes before the next one starts. This avoids the complexity of managing multiple transactions at once but can result in higher latency for some operations.
- **Two-Phase Transfer:** APB transactions are carried out in two phases: the setup phase and the enable phase. This makes it simple to manage the flow of data between the master and slave.
- **Single Clock Domain:** All APB transactions operate within a single clock domain, which simplifies timing and reduces issues related to clock domain crossings.
- **Wait States:** The protocol allows the slave to introduce wait states if it cannot complete the transaction within a single clock cycle, by asserting the PREADY signal.

3.2 APB Protocol Signals

The APB protocol uses the following signals to control and manage data transfers. Each of these signals is essential to ensuring reliable communication between the standard and used signals in this project. These signals ensure smooth and efficient data transfer, maintaining proper synchronization and communication integrity. The proper management of these signals is crucial to achieving a stable and functional APB protocol implementation in the system, as listed below:

Signal	Description
PCLK	Clock signal for the APB system. All transactions are synchronized to this clock.
PRESETn	Active-low reset signal. When PRESETn is low, it resets all APB peripherals.
PADDR	Address bus. Specifies the address in the slave device where data is read or written.
PSEL	Peripheral select signal. Selects a specific peripheral slave for the transaction.
PWRITE	Write control signal. Indicates if the operation is a write (PWRITE = 1) or a read (PWRITE = 0).
PWDATA	Write data bus. Carries data to be written to the selected slave peripheral.
PRDATA	Read data bus. Carries data returned from the slave peripheral to the master.
PENABLE	Enable signal. Used to transition from the setup phase to the enable phase.
PREADY	Ready signal. Indicates if the slave is ready to complete the transaction.
PSLVERR	Slave error signal. Indicates that an error occurred during the transfer.

Table 3.1: APB Protocol Signals

3.3 Detailed Explanation of Each Signal

3.3.1 PCLK (Clock)

Description:

- The PCLK signal is the primary clock for the APB system.
- Every signal in APB is synchronized to this clock.
- All transactions, including address setup, data transfer, and response, occur relative to the rising edge of PCLK.

Importance:

- It ensures that the entire APB bus operates in sync.

- It determines the timing of each operation.

Behavior:

- All other signals are sampled on the rising edge of PCLK.

3.3.2 PRESETn (Active-low Reset)

Description:

- This signal is used to reset the APB bus.
- When PRESETn = 0 (active low), the APB peripherals are reset to their initial state.

Importance:

- It ensures that the system starts in a known state before any transactions can take place.
- Resetting peripherals to a default state is critical to prevent erroneous behavior.

Behavior:

- Typically, the reset is active for one or more clock cycles at startup.
- Once PRESETn is released (set to high), normal operation resumes.

3.3.3 PADDR (Address Bus)

Description:

- This signal carries the address of the register or memory location in the selected slave device that the master wishes to access.
- The width of PADDR is typically determined by the address space of the slave devices.

Importance:

- It allows the master to specify the target of the transaction.

Behavior:

- During the setup phase, the master drives a valid address on PADDR.

3.3.4 PSEL (Peripheral Select)

Description:

- The PSEL signal is used to select the slave peripheral for the transaction.
- If $PSEL = 1$, the specified slave is selected and will participate in the transaction.
- If $PSEL = 0$, no transaction occurs.

Importance:

- It ensures that the correct peripheral is involved in the current transaction.

Behavior:

- It is driven high by the master to indicate that the peripheral is being selected.

3.3.5 PWRITE (Write Control Signal)

Description:

- The PWRITE signal indicates whether the current operation is a write ($PWRITE = 1$) or a read ($PWRITE = 0$).

Importance:

- It tells the slave whether to store data (in the case of a write) or to retrieve data (in the case of a read).

Behavior:

- It is asserted by the master to specify the direction of data flow.

3.3.6 PWDATA (Write Data Bus)

Description:

- The PWDATA signal carries the data to be written to the selected slave.
- This signal is only valid during a write operation.

Importance:

- It ensures that the correct data is transmitted to the slave.

Behavior:

- This signal is driven by the master during a write transaction, and the slave receives it during the enable phase.

3.3.7 PRDATA (Read Data Bus)

Description:

- The PRDATA signal carries the data that the slave returns to the master during a read operation.

Importance:

- It provides the master with the requested data from the slave.

Behavior:

- This signal is driven by the slave during a read operation, and the master receives it during the enable phase.

3.3.8 PENABLE (Enable Signal)

Description:

- The PENABLE signal is used to transition from the setup phase to the enable phase of a transaction.
- When PENABLE is high, it indicates that the data transfer is in progress, and valid data (either to write or read) should be transferred.

Importance:

- It indicates when the master has completed setting up the transaction and when the data can actually be transferred.

Behavior:

- It is asserted by the master after the setup phase and deasserted after the transaction completes.

3.3.9 PREADY (Ready Signal)

Description:

- The PREADY signal is the slave's way of indicating whether it is ready to complete the transaction.
- If PREADY = 1, the slave is ready, and the transfer proceeds.

- If $PREADY = 0$, the master must wait (this introduces a wait state).

Importance:

- It allows the slave to delay the transfer if it requires more time to complete an operation, ensuring that the transaction is completed without errors.

Behavior:

- It is asserted by the slave during the enable phase and can be held low for multiple clock cycles to introduce wait states.

3.3.10 PSLVERR (Slave Error Signal)

Description:

- The PSLVERR signal is an optional signal that the slave can assert to indicate that an error has occurred during the transaction.
- This could happen if the slave encounters an issue, such as an invalid address or an unsupported operation.

Importance:

- It allows the slave to communicate to the master that something went wrong, which the master should handle (such as aborting the transaction).

Behavior:

- It is typically used in more complex designs where peripheral devices might need to handle errors (e.g., a memory write failure).

3.4 APB Timing Diagrams with Wait States and Error Handling

3.4.1 Write Operation Without Wait States

In a write operation, the master initiates the transaction by driving the address (PADDR) and write data (PWDATA). When the master asserts PENABLE, the slave completes the write operation if it is ready ($PREADY = 1$).

Timing Diagram (Write Operation Without Wait States)

In this example, $PREADY$ is high ($PREADY = 1$), meaning the slave is ready to complete the transaction in one cycle.

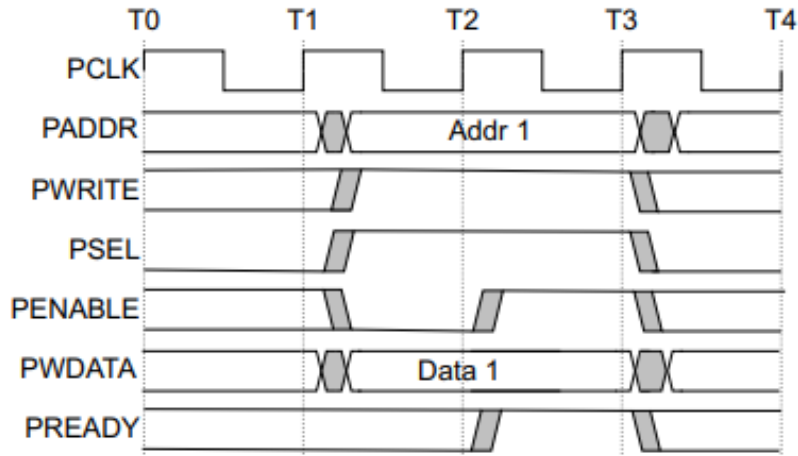


Figure 3.1: Write Operation Without Wait States

3.4.2 Write Operation With Wait States

In the case of wait states, the slave is not ready to complete the transaction immediately. The master must wait until the slave asserts $PREADY = 1$ after several cycles.

Timing Diagram (Write Operation With Wait States)

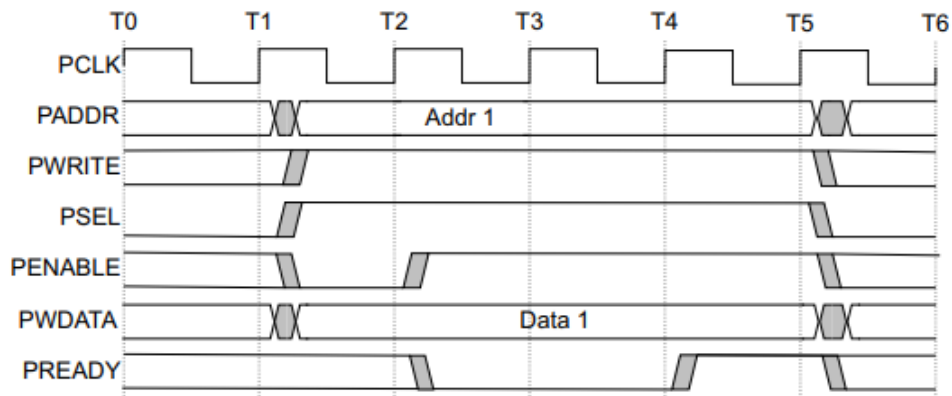


Figure 3.2: Write Operation With Wait States

In this case, the slave holds $PREADY = 0$ for one cycle (introducing a wait state), and the master must wait for $PREADY = 1$ before the transfer is complete.

3.4.3 Read Operation Without Wait States

In a read operation, the master provides the address (PADDR) and asserts PSEL to select the slave. When the slave is ready, it drives the data (PRDATA) on the bus and asserts $PREADY = 1$.

Timing Diagram (Read Operation Without Wait States)

The master receives the data from the slave (PRDATA) when $PREADY = 1$.

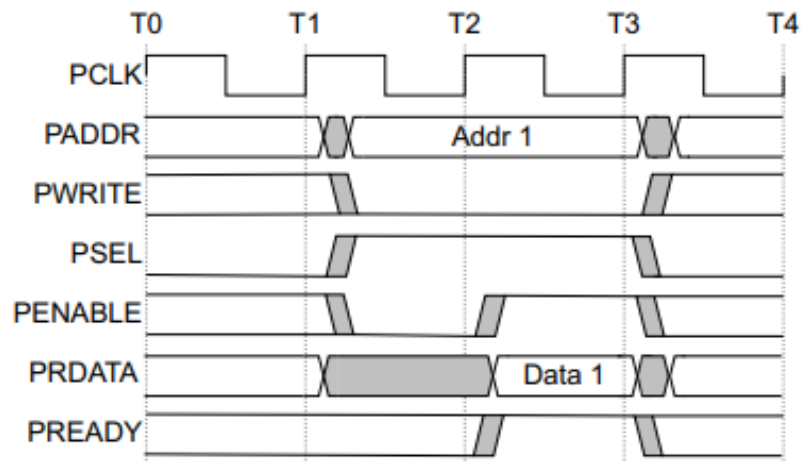


Figure 3.3: Read Operation Without Wait States

3.4.4 Read Operation With Wait States

In a read operation with wait states, the slave delays the data transfer by holding $PREADY = 0$ for one or more cycles.

Timing Diagram (Read Operation With Wait States)

In this case, the slave holds $PREADY = 0$ for one cycle, forcing the master to wait. The data is only available when $PREADY = 1$.

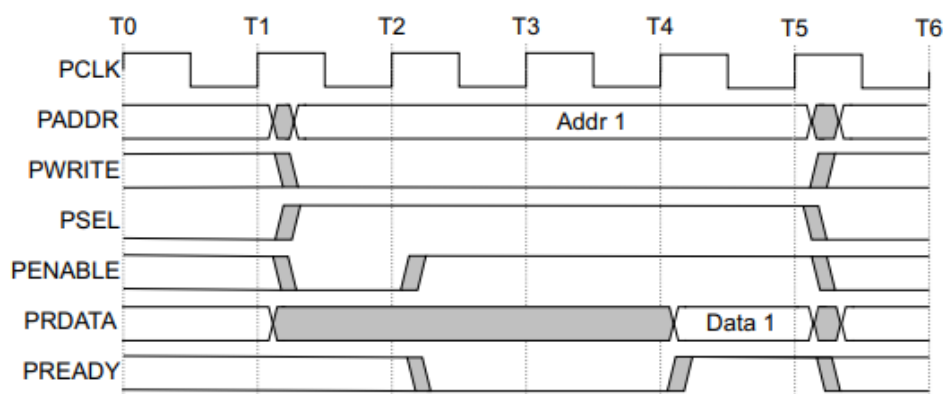


Figure 3.4: Read Operation With Wait States

3.4.5 Error Handling (PSLVERR)

The PSLVERR signal is used to indicate that an error has occurred during a transaction. If the slave encounters an invalid address, unsupported operation, or another issue, it asserts PSLVERR = 1 to signal the error to the master. The master can handle the error by aborting the transaction or taking corrective action.

Error State Timing Diagram (Write Operation with Error)

Here, PSLVERR = 1 indicates that an error occurred during the transaction (perhaps an invalid address), signaling the master that it must take corrective action, such as retrying or aborting the transaction.

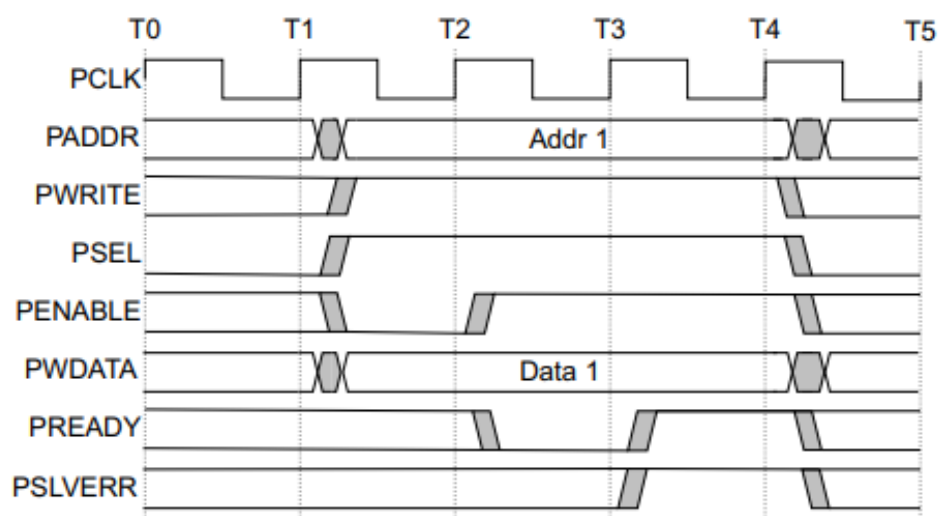


Figure 3.5: Write Operation with Error

Chapter 4: Reference Architecture of APB Protocol

4.1 Overview of APB Architecture

The Advanced Peripheral Bus (APB) is a part of the AMBA (Advanced Microcontroller Bus Architecture) family developed by ARM. It is optimized for minimal power consumption and low complexity, designed primarily for communication with low-bandwidth peripherals in embedded systems.

APB is a non-pipelined, single-clock, and synchronous protocol. It operates on a simple two-phase protocol — setup and access — and supports only one master. Unlike AXI or AHB buses, APB does not support burst or pipelined transfers, making it suitable for simple register-based peripherals.

Key features of the APB architecture include:

- Simple, low power interface.
- Non-pipelined data transfer.
- No need for arbitration.
- Supports read and write operations.
- Control via simple signals: PSELx, PENABLE, PWRITE, PREADY, and PRDATA.

4.2 Master-Slave Communication Flow

The Advanced Peripheral Bus (APB) follows a master-slave communication architecture. The master initiates and controls all bus transactions, while the slave responds to the master's requests. APB is designed to provide a low-complexity interface for connecting peripherals that do not require high performance.

Figure 4.1 illustrates a typical reference architecture of the APB protocol. The system includes a **Top Module** containing the Master and Slave blocks.

- **Top Module:** It encapsulates both the master and the slave and acts as a wrapper connecting the external signals (like clock and reset) to internal APB signals.

- **Master:** This block generates control and address signals and initiates either read or write operations.
- **Slave:** This block receives signals from the master, decodes them, and performs the necessary memory or peripheral operations. It responds with data or ready/error status.

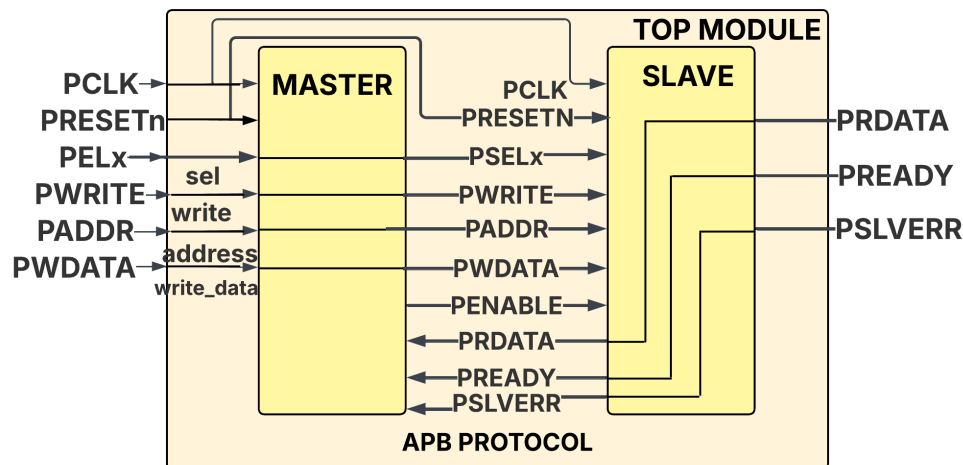


Figure 4.1: Reference Architecture of APB Protocol

Master Inputs and Outputs

- PCLK – Clock input for synchronization.
- PRESETn – Active-low reset for initializing internal state.
- PSELx – Slave select signal. High when the corresponding slave is selected.
- PWRITE – Determines operation type: Write (1) or Read (0).
- PADDR – Address bus carrying the address of the target slave register.
- PWDATA – Data to be written during a write operation.
- PENABLE – Control signal to enable the operation during ACCESS phase.
- PRDATA – Data received from the slave during a read operation.
- PREADY – Indicates the slave is ready to complete the current operation.
- PSLVERR – Error flag from the slave indicating unsuccessful transfer.

Slave Inputs and Outputs

- PSELx – Received from the master to indicate selection.
- PWRITE, PADDR, PWDATA – Control, address, and data inputs from the master.
- PENABLE – Received from the master to trigger the transfer.
- PRDATA – Data output for read operations.
- PREADY – Indicates to the master that the slave is ready.
- PSLVERR – Error signal if the transaction cannot be completed.

The master uses the above signals to initiate and control all transfers. The slave, once selected, acknowledges by asserting PREADY. In the event of an error, it asserts PSLVERR to inform the master.

4.3 Timing and Control Signals in Reference Design

The APB protocol operates with a well-defined and predictable timing model, making it suitable for low-speed peripherals. Each transaction consists of two distinct phases: the **SETUP** phase and the **ACCESS** phase. These are governed by a simple finite state machine (FSM), as shown in Figure 4.2.

State Machine Phases

- **IDLE:** No transfer is active. The default state of the FSM. Signals like PSELx and PENABLE are deasserted.
- **SETUP:** Initiates the transfer. The master drives the address, control, and data lines. PSELx is asserted, PENABLE remains low.
- **ACCESS:** The slave completes the read/write. PENABLE is asserted. If PREADY is high, the transfer is completed.

Control Signal Descriptions

- PCLK – Clock input driving all sequential logic in master and slave.
- PRESETn – Active-low asynchronous reset to bring FSM into IDLE.
- PSELx – Used to select the slave for communication.

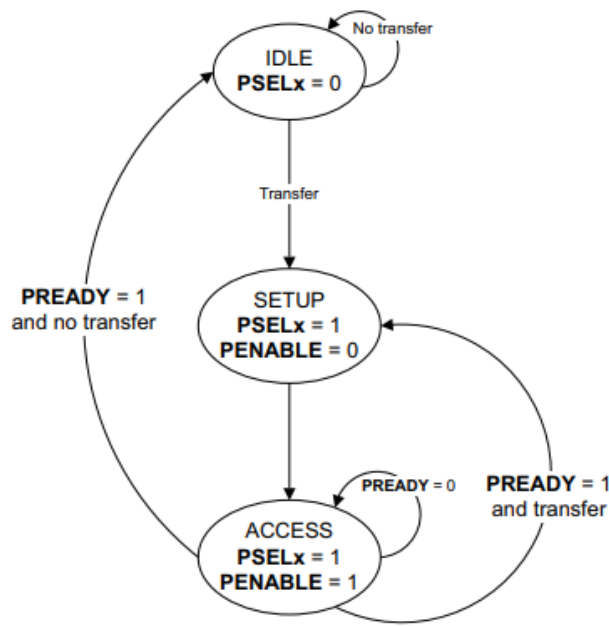


Figure 4.2: State Diagram of APB Protocol

- **PENABLE** – Indicates the start of the ACCESS phase.
- **PWRITE** – Determines if the operation is a read (0) or write (1).
- **PADDR** – Carries address of the target slave register.
- **PWDATA** – Write data from master to slave.
- **PRDATA** – Read data from slave to master.
- **PREADY** – Slave asserts high when it is ready to complete transaction.
- **PSLVERR** – Error signal. High indicates an invalid transfer.

4.3.1 APB Signal Timing Behavior

Timing Considerations

All signals are synchronized to PCLK. The slave should not change any output signal until PENABLE is asserted. A read operation is only valid when PREADY is high in the ACCESS phase. If PREADY remains low, the master waits in the ACCESS state, ensuring handshake synchronization between master and slave.

Signal	Description
PCLK	Clock signal from the APB system
PRESETn	Active-low reset signal
PSEL	Peripheral select; enables this slave
PENABLE	Indicates the access phase of the transaction
PWRITE	1 = write, 0 = read
PADDR	8-bit address to access 256 locations
PWDATA	8-bit input data bus (used during write)
PRDATA	8-bit output data bus (used during read)
PREADY	Always high; indicates zero wait-state response

Table 4.1: APB Signal Timing Behavior

This timing mechanism offers a balance between simplicity and efficiency, which makes the APB protocol well-suited for integrating low-bandwidth peripherals into SoC designs.

4.4 APB Bus Interconnect Structure

The interconnect in an APB system consists of:

- One APB master (usually a bridge from AHB/AXI)
- Multiple APB slaves (peripherals)
- Decoder logic to assert only one PSELx

Each slave is memory-mapped and responds only when selected. The bus architecture avoids arbitration since there is only one master. Shared buses carry control and data signals, while only the selected slave drives response signals back to the master.

This simple interconnect model allows easy integration of multiple peripherals, making APB suitable for systems-on-chip (SoC) designs where minimal logic and deterministic timing are critical.

- High-bandwidth memory interface: Facilitates rapid data transfer between memory and the processor.
- High-performance ARM processor: Acts as the central unit for executing instructions and managing tasks.

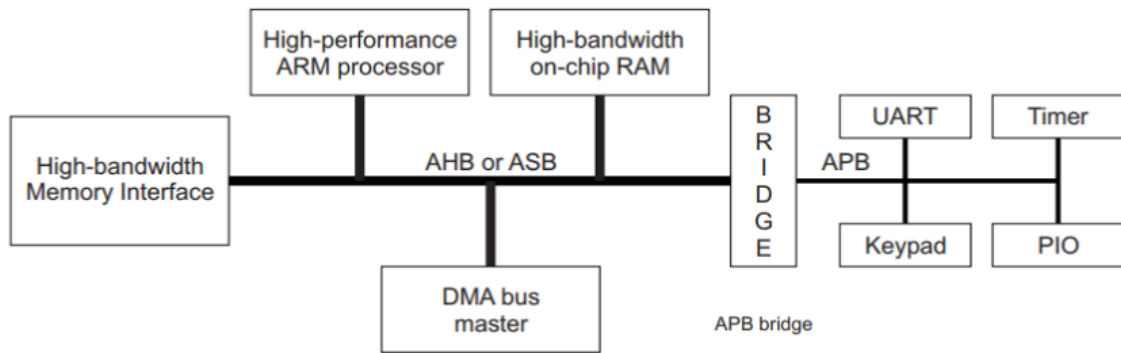


Figure 4.3: APB Interconnect architecture

- High-bandwidth on-chip RAM: Provides quick access to frequently used data, enhancing system performance.
- DMA bus master: Enables direct memory access, connected via AHB or ASB bus, reducing CPU load.
- APB bridge: Connects peripherals like UART, keypad, PIO, and timer through the APB bus, ensuring smooth communication.

Chapter 5: RTL Design and Simulation

This chapter details the RTL design and simulation phase of the APB protocol implementation. It includes the overall design methodology, coding of the master and slave modules based on protocol behavior, testbench creation for validation, and the results obtained from functional simulation. The foundation of this work was built upon the architecture and state machine discussed in Chapter 4.

5.1 Design Flow

The RTL design process followed in this project is aligned with industry-standard hardware development methodology and builds upon the reference APB architecture described in Chapter 4. This includes the master-slave interconnect, data/control signal mapping, and modular hierarchy.

The step-by-step flow is as follows:

- **Architecture Reference and Planning:** The APB system-level architecture was derived from earlier research and practical references (see Chapter ??). This includes a central APB master communicating with one or more slaves via shared data, address, and control lines.
- **Design Specification and Interface Finalization:** Signal specifications such as PWRITE, PSEL, PENABLE, PRDATA, PWDATA, PREADY, and PSLVERR were finalized with clear roles and timing definitions.
- **Implementation Diagram:** A dedicated APB implementation diagram was generated to reflect the RTL structure. It captures the internal connection between the APB master and slaves, along with signal directions and interface boundaries.
- **RTL Module Development:** Using the planned interface and diagrams, RTL coding was performed (described in Section 5.2).
- **Testbench and Simulation Setup:** A test environment was created to apply dynamic stimuli and observe functional behavior.
- **Analysis and Debugging:** Functional simulation was used to verify timing, logical correctness, and protocol compliance (see Section 5.4).

5.2 RTL Coding of APB Modules

The RTL development was guided by the APB master state diagram previously discussed in Chapter ???. This state machine governed the sequencing of control signals during APB transactions and was central to ensuring protocol compliance.

APB Master Design

The master module was developed to:

- Initiate read and write transactions using the state-driven approach
- Generate and manage PSEL, PENABLE, and PWRITE
- Control the flow of PADDR and PWDATA
- Handle responses via PREADY and PSLVERR

The design strictly followed the two-phase protocol: the setup phase and the enable phase. Transitions were governed by the internal state machine explained in Chapter ??.

APB Slave Design

The slave module responded to control signals from the master and supported both read and write operations.

An important feature was the ability to detect and signal invalid address accesses. The valid address range was defined from 0 to 170 (decimal). Accesses outside this range generated a slave error by asserting the PSLVERR signal.

In such cases:

- No data transfer occurred.
- PRDATA was set to a default/undefined value.
- PREADY was still asserted to end the cycle.

This behavior ensures graceful handling of exceptions and reflects real-world bus error mechanisms used in APB-based systems.

5.3 Testbench Development

A comprehensive testbench was developed to validate protocol compliance and functional correctness. Test scenarios included basic functionality, stress cases, error handling, and reset conditions. Dynamic inputs were provided using `$value$plusargs`.

1. **Sanity Test:** Write to an address and read back the value.
2. **Back-to-Back Write and Read:** Continuous write-read cycles without idle gaps.
3. **Burst (5x) Write and Read:** Sustained operation test over multiple addresses.
4. **Asynchronous Reset Between Operations:** Verifies recovery after mid-operation reset.
5. **Invalid Address Access:** Slave error generation by addressing outside 0–170.

Each testcase included logging and waveform tracing for detailed post-simulation analysis.

5.4 Simulation Results

Simulations were performed using [Insert Simulator Name]. Below are waveform snapshots and observations for each testcase.

1. Sanity Test

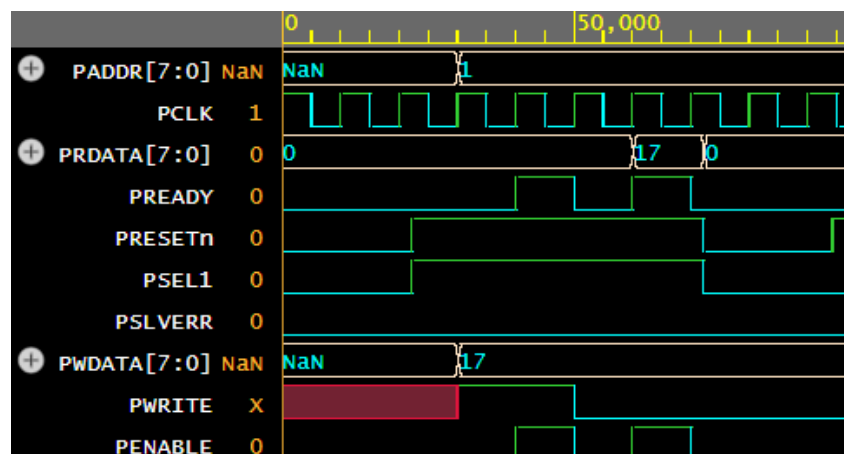


Figure 5.1: Waveform for Sanity Test (Write and Read Same Address)

Result: Correct data read. Protocol handshaking verified.

2. Back-to-Back Write and Read

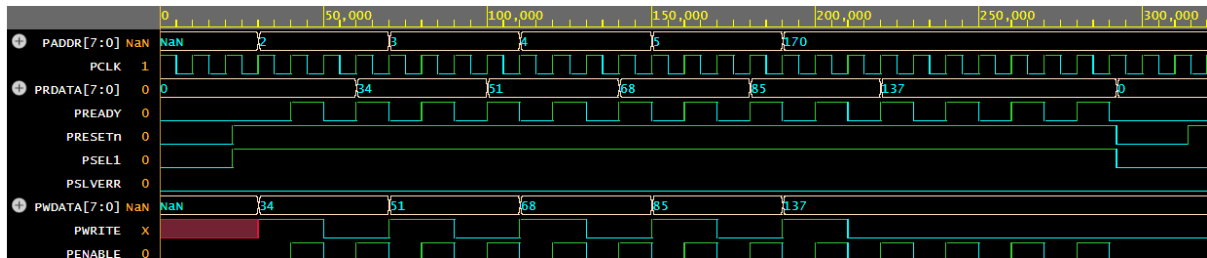


Figure 5.2: Waveform for Back-to-Back Write and Read

Result: Seamless operation without idle cycles. All transactions acknowledged.

3. Burst Write and Read (5 Times)

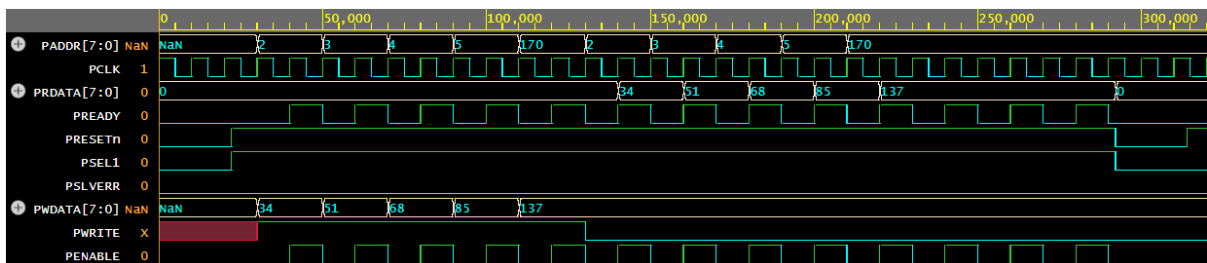


Figure 5.3: Waveform for 5x Write and Read Burst

Result: All writes and reads succeeded. Data matched expected values.

4. Asynchronous Reset Between Operations

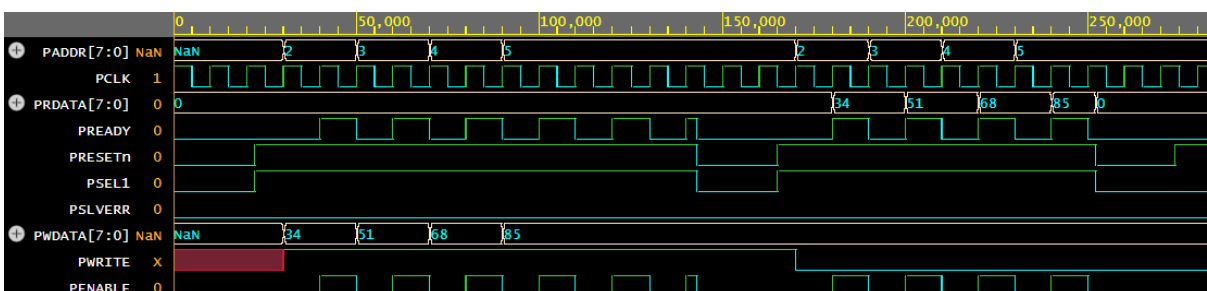


Figure 5.4: Waveform Showing Asynchronous Reset in Middle of Transaction

Result: Reset acknowledged. Protocol resumed correctly after reset.

5. Invalid Address Access

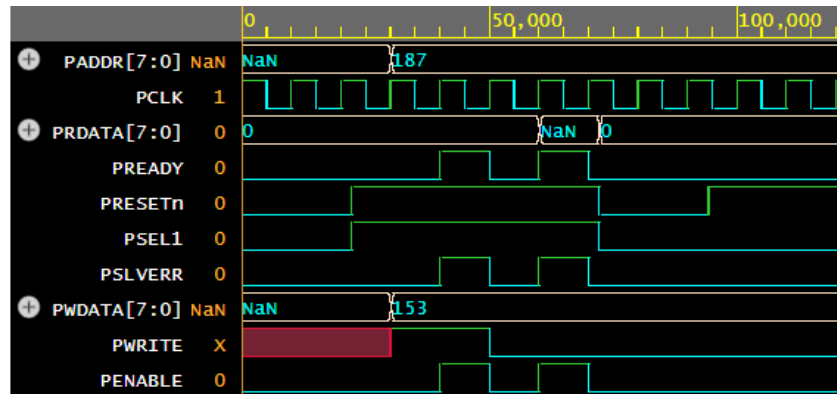


Figure 5.5: Waveform Showing Slave Error on Invalid Address Access

Result: PSLVERR correctly asserted. No data corruption occurred.

Chapter 6: Slave Devices Design and Selection

In the Advanced Peripheral Bus (APB) protocol, slave devices are peripherals that respond to read and write operations initiated by the APB master. These devices operate with relatively low bandwidth requirements and are connected using a simple, low-latency interface.

This chapter provides a detailed explanation of the design and implementation of three APB-compliant slave peripherals: a memory module, a 7-segment display driver, and a UART communication interface. Each slave was implemented using Verilog HDL and tested on the FPGA hardware platform.

6.1 Slave Selection

In this APB-based system, the master communicates with three slave devices: UART, 7-Segment Display, and Memory. Slave selection is explicitly handled through a 2-bit control signal named PSELx, which is generated by the APB master. Each unique value of PSELx corresponds to a specific slave device, ensuring that only one slave is active during any transaction. The architecture is illustrated in the figure below:

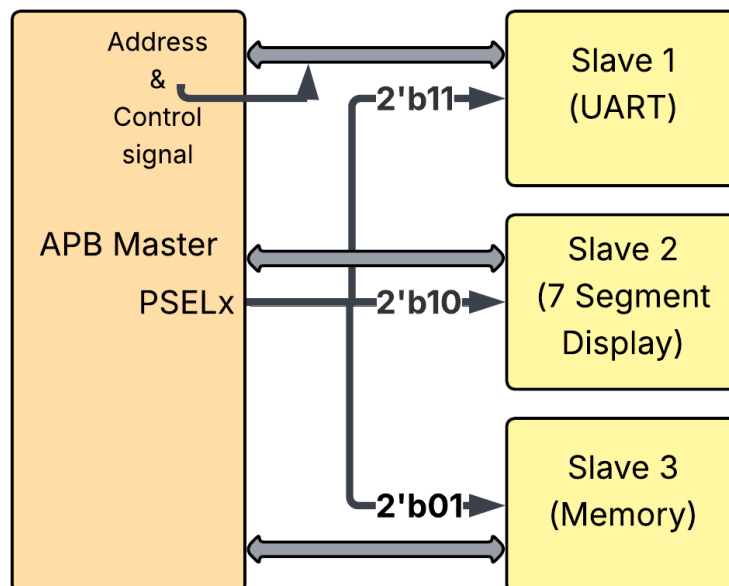


Figure 6.1: APB Master with Three Slaves (UART, 7-Segment Display, Memory)

Slave Mapping Using PSELx:

PSELx Value	Selected Slave
2'b11	Slave 1 – UART
2'b10	Slave 2 – 7-Segment Display
2'b01	Slave 3 – Memory

Table 6.1: Slave Selection

In this design, PSELx is not derived from address bits (PADDR). Instead, it is directly controlled by the APB master to target a specific slave. This manual slave-selection approach simplifies the APB master logic and is effective in small-scale designs involving a few peripherals.

Each slave module internally monitors the PSELx signal. When its assigned 2-bit value appears on PSELx, that slave is activated and begins responding to control signals such as PENABLE, PWRITE, PWDATA, or PRDATA.

This method offers the following advantages:

- Simplified slave interface logic (no internal decoding)
- Easier debugging and testbench implementation
- Clear, fixed mapping between master control and slave selection

6.2 Memory Slave

6.2.1 Objective

The purpose of designing the Memory Slave is to implement a simple APB-compatible read-write memory module that supports both read and write operations. This slave acts as a Random Access Memory (RAM) block, accessible by the APB master through a defined address and data bus. It provides temporary storage functionality and supports bidirectional data access under control of the APB protocol.

6.2.2 Design Overview

The memory slave is implemented as a RAM with a total of 256 memory locations, each capable of storing 8-bit wide data. The design supports full read and write functionality, allowing the APB master to write data to any location and read it back during subsequent transactions.

Key Parameters:

- **Memory Type:** RAM (Random Access Memory)
- **Depth:** 256 (8-bit address bus)
- **Word Size (Data Width):** 8 bits
- **Access Type:** Read and Write
- **Interface:** APB (AMBA 3 APB Compliant)

6.2.3 Functional Description

The RAM contains 256 registers indexed by an 8-bit address line (PADDR). The APB master performs write operations by asserting the PWRITE signal high and placing the target address and data on the PADDR and PWDATA lines, respectively. Read operations are performed when PWRITE is low, and the selected data is driven on the PRDATA bus.

APB Write Operation Flow:

- PSEL = 1: Slave is selected by the APB master.
- PENABLE = 1: Indicates the enable/access phase.
- PWRITE = 1: Specifies a write operation.
- PADDR: Target address for data storage.
- PWDATA: 8-bit data to be stored.
- **Memory Update:** Data is written to the memory location addressed by PADDR.

APB Read Operation Flow:

- PSEL = 1: Slave is selected by the APB master.
- PENABLE = 1: Indicates the enable/access phase.
- PWRITE = 0: Specifies a read operation.
- PADDR: Address of the location to read from.
- PRDATA: Data at the given address is driven onto the read bus.

The memory slave is always ready to respond (i.e., PREADY = 1), and there are no wait states in this implementation.

6.2.4 Verilog HDL Implementation Overview

The RAM is implemented as a register array of size 256×8. Write operations are synchronous and performed on the rising edge of the clock when PWRITE, PSEL, and PENABLE are all asserted. Read operations are performed combinatorially, outputting the selected data on PRDATA whenever a valid read transaction is detected.

Although the write operation is clocked, the read operation is combinational to allow faster access within a single APB cycle.

6.2.5 APB Signal Timing Behavior

Signal	Description
PCLK	Clock signal from the APB system
PRESETn	Active-low reset signal
PSEL	Peripheral select; enables this slave
PENABLE	Indicates access phase of the transaction
PWRITE	1 = write, 0 = read
PADDR	8-bit address to access 256 locations
PWDATA	8-bit input data bus (used during write)
PRDATA	8-bit output data bus (used during read)
PREADY	Always high; indicates zero wait-state response

Table 6.2: APB Signal Timing Behavior

6.2.6 Simulation and Testing

The memory slave was tested by executing a sequence of write operations followed by read operations to verify data integrity. The following simulation scenario was used to validate behavior:

Write Test Case:

- PADDR = 0x10, PWDATA = 0x5A, PWRITE = 1 → Store 0x5A at address 0x10
- PADDR = 0x20, PWDATA = 0xC3, PWRITE = 1 → Store 0xC3 at address 0x20

Read Test Case:

- PADDR = 0x10, PWRITE = 0 → Output: PRDATA = 0x5A
- PADDR = 0x20, PWRITE = 0 → Output: PRDATA = 0xC3

Correct values were returned during read operations, confirming that the memory was written to and read from successfully using the APB interface.

6.3 7-Segment Display

A typical 7-segment display consists of seven LEDs (light-emitting diodes) arranged in a rectangular fashion. Each LED is referred to as a segment because when illuminated, it forms part of a numeral (0-9) or letter. The segments are labeled as 'a' through 'g', and an additional segment called 'dp' represents the decimal point.

6.3.1 Working Principle

The Figure 6.2 illustrates the connection of a common anode 7-segment display with current limiting resistors. In this configuration:

- The common anode pin is connected to the positive voltage supply (+Vcc).
- Each segment (a-g and dp) has its cathode connected through individual current limiting resistors (220Ω each) to ground (0V).

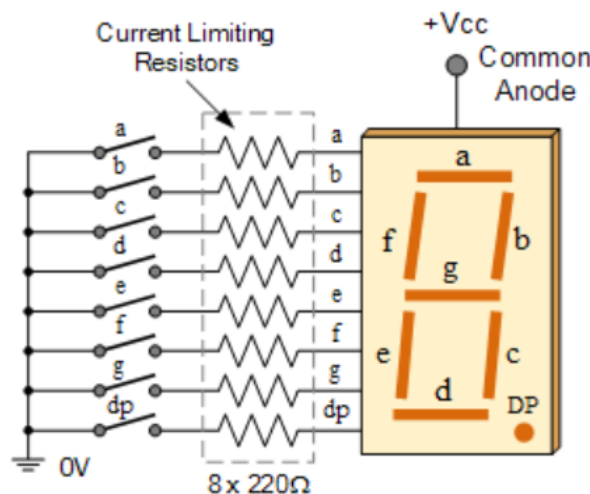


Figure 6.2: anode configuration

When a specific combination of segments is illuminated by applying voltage across their respective pins, different numerals or characters are displayed.

6.3.2 Hexadecimal Values Representation

The table below shows how hexadecimal values correspond to the segments that need to be lit up on the 7-segment display:

Hex Value	Segments Lit	Binary Representation
0	abcdef	0111111
1	bc	0000110
2	abdeg	1011011
3	abcdg	1001111
4	bcfg	1100110
5	acdfg	1101101
6	acdefg	1111101
7	abc	0000111
8	abcdefg	1111111
9	abcdfg	1101111
A	abcefg	1110111
B	cdefg	1111100
C	adef	0111001
D	bcdeg	1011110
E	adefg	1111001
F	aefg	1110001

Table 6.3: Hexadecimal values and their corresponding segments

6.3.3 Connecting a 7-Segment Display to APB

The interface between the 7-segment display and the APB (Advanced Peripheral Bus) protocol involves reading and writing data to control the display. The APB protocol uses control signals such as PWRITE, PADDR, PWDATA, PRDATA, and PENABLE to manage data transactions between the master and slave devices.

Write Operation (Master writes data to 7-segment display)

1. The master initiates a write operation by setting the PWRITE signal high, indicating that data will be written. 2. The PADDR signal specifies the address of the 7-segment display's data register. 3. The PWDATA signal carries the data (e.g., a digit) that the master wants to display. 4. The PENABLE signal is activated, and the slave receives the data from PWDATA. 5. The display updates according to the received data, showing the corresponding digit.

Write Operation: PWRITE = 1, PADDR, PWDATA, PENABLE

Read Operation (Master reads data from 7-segment display)

1. If the master wants to read the current value or status of the display, it initiates a read operation by setting PWRITE low. 2. The PADDR signal specifies the address of the register to be read (such as a status or value register). 3. The slave places the requested data on the PRDATA signal. 4. When PENABLE is active, the master reads the data from PRDATA.

Read Operation: PWRITE = 0, PADDR, PENABLE, PRDATA

6.4 UART

UART (Universal Asynchronous Receiver/Transmitter) is a widely used serial communication protocol that enables data transfer between devices using just two wires: Tx (transmit) and Rx (receive). It converts parallel data into serial form for transmission and vice versa at the receiver's end.

Unlike synchronous protocols, UART is asynchronous—no shared clock is needed. Instead, data frames are marked by a *start bit* at the beginning and one or more *stop bits* at the end. These framing bits help synchronize the data between sender and receiver.

Accurate data transmission depends on both UART devices being configured with the same **baud rate**, data bits, parity bit, and stop bits. A mismatch can cause communication errors.

Despite its simplicity, UART provides reliable and efficient serial communication and is commonly used in embedded systems for its low hardware overhead and flexibility.

6.4.1 How UART Works

In UART communication, the transmitting UART receives parallel data from a data bus connected to devices like a CPU or microcontroller. It then forms a data packet by adding framing bits—*start bit*, optional *parity bit*, and one or more *stop bits*.

This packet is transmitted serially via the Tx pin. The receiving UART reads the serial data through its Rx pin, removes the framing bits, converts the data back to parallel form, and sends it to the receiving device's data bus. This enables reliable and efficient serial communication between devices.

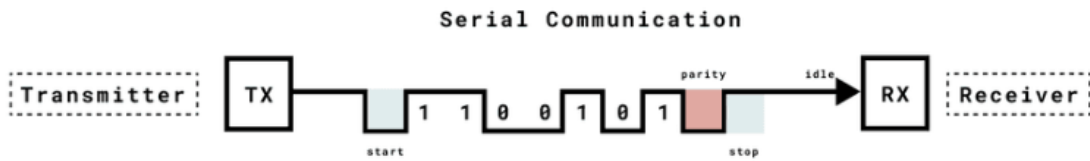


Figure 6.3: UART Serial Communication

6.4.2 Frame Format

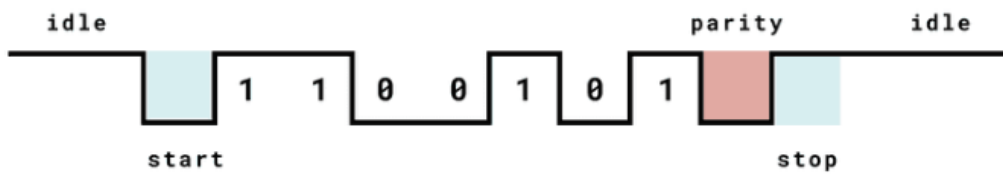


Figure 6.4: UART Frame Format

6.4.3 Start Bit

Start Bit: Always logic 0. Indicates the beginning of a frame.

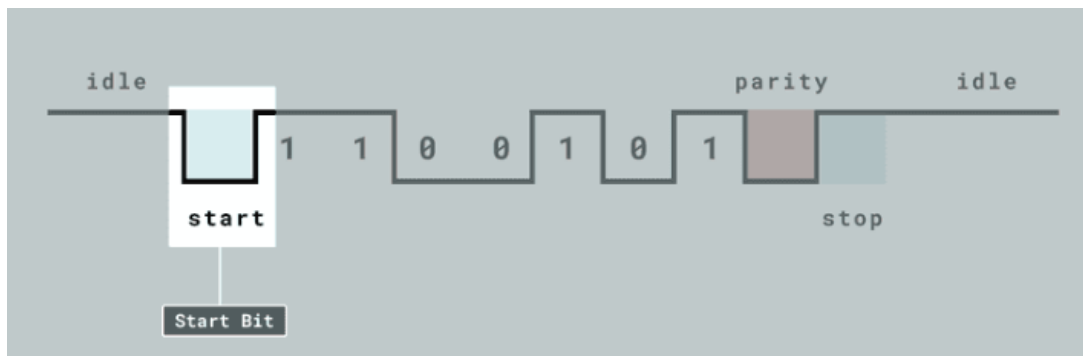


Figure 6.5: UART Start bit

6.4.4 Data bit

Data Bits: Usually 8 bits. Carry actual information.

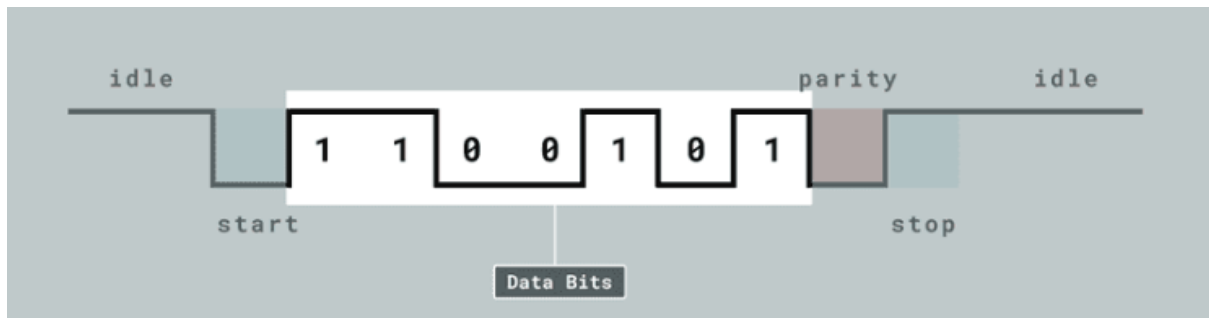


Figure 6.6: UART Data bit

6.4.5 Parity bit

Parity Bit: Optional. Used for error detection.

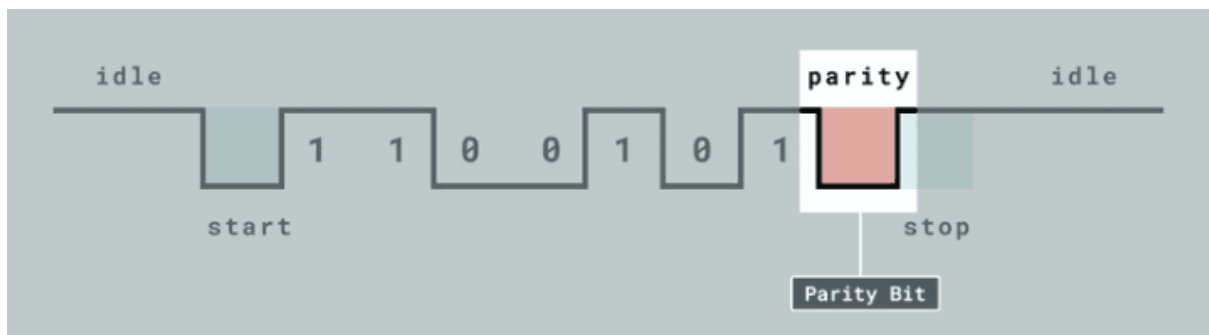


Figure 6.7: UART Parity bit

6.4.6 Stop bit

Stop Bit(s): One or two logic 1 bits to mark frame end.

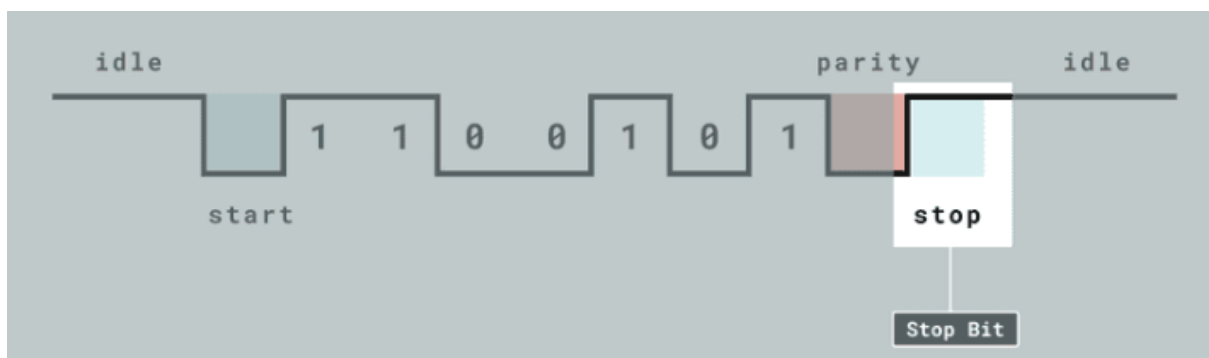


Figure 6.8: UART Stop bit

6.4.7 UART Transmitter Module (Summary)

The UART transmitter module accepts three inputs: clock signal (`clk`), 8-bit data (`data`), and a transmission enable signal (`en_tx`). It produces two outputs: the serial transmission signal (`u_tx`) and a flag (`u_tx_done`) indicating the completion of transmission. The module functions as a finite state machine (FSM) with the following states:

- **IDLE:** Waits for `en_tx` signal to become high. Output remains in high-impedance state.
- **START:** Initializes data and counter registers, transmits start bit (logic 0).
- **DATA:** Serially transmits 8 data bits using a bit counter.
- **PARITY:** Computes and sends even parity bit using XOR.
- **DONE:** Sets `u_tx_done = 1`, then returns to START for the next transmission.

All state transitions are triggered on the falling edge of the clock signal (`negedge clk`).

6.4.8 UART Receiver Module (Summary)

The UART receiver module takes clock input (`clk`), serial data input (`u_rx`), and an enable signal (`en_rx`). It outputs the received 8-bit data (`data`) and a completion flag (`u_rx_done`). The FSM in the receiver progresses through the following states:

- **IDLE:** Waits for `en_rx` to be high and detects a start bit (`u_rx = 0`).
- **DATA:** Captures 8 bits from `u_rx` into an internal register.
- **PARITY:** Computes expected parity using XOR and compares with received parity bit.
- **DONE:** If parity is correct, moves data to `data` output and sets `u_rx_done = 1`.

The module uses `negedge clk` for sequential logic, ensuring accurate bit sampling and control flow.

Chapter 7: FPGA Implementation

7.1 FPGA Board Description: Artix-7 Nexys 4

In this project, the **Nexys 4 FPGA development board** shown in FIGURE 7.1, based on the **Xilinx Artix-7 FPGA (XC7A100T-1CSG324C)**, is used for the design and implementation of the APB (Advanced Peripheral Bus) protocol. The Nexys 4 board is a powerful and versatile platform ideal for digital system design and prototyping, widely used in academic and industrial applications.

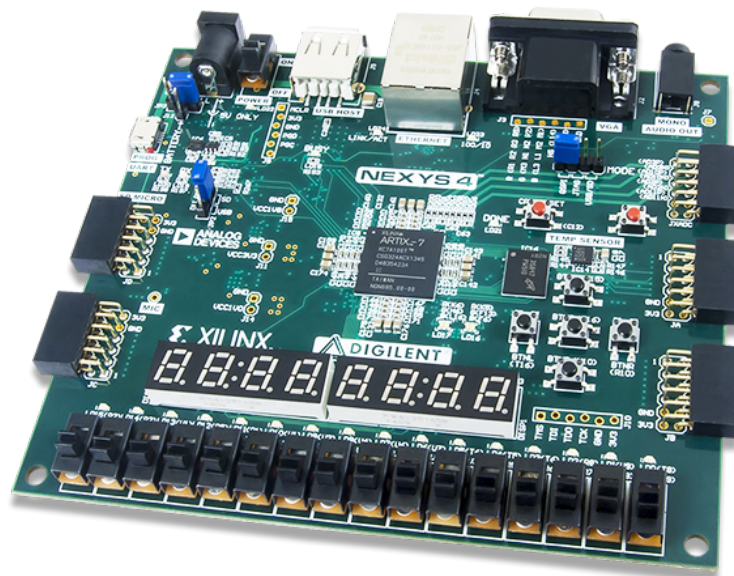


Figure 7.1: Artix-7 Nexys 4 FPGA Board

7.1.1 Overview of Nexys 4

The Nexys 4 board is developed by **Digilent Inc.** and features the **Xilinx Artix-7 FPGA**, which is optimized for cost and performance. The board provides a wide range of I/O options, memory interfaces, and built-in peripherals that support real-time system development, communication interfaces, and high-speed digital designs.

The Artix-7 family delivers high system performance while maintaining lower power consumption and cost compared to previous FPGA generations. The board is fully supported by the Xilinx Vivado Design Suite, which provides an integrated development environment for synthesis, implementation, and programming.

7.1.2 Key Features of Nexys 4 Board

- **FPGA Chip:** Xilinx Artix-7 (XC7A100T-1CSG324C)
 - Logic Cells: 101,440
 - Flip-Flops: 126,800
 - Block RAM: 4,860 Kbits
 - DSP Slices: 240
 - Operating Voltage: 1.0V core
- **Onboard Memory:**
 - 16MB QSPI Flash for configuration storage
 - 128MB DDR2 SDRAM
- **I/O Features:**
 - 16 Slide Switches
 - 16 LEDs
 - 5 Push Buttons
 - 4-digit Seven-Segment Display
 - 12-bit VGA output
 - Pmod Connectors for peripheral expansion
- **Communication Interfaces:**
 - USB-JTAG for Programming & Debugging
 - USB-UART Bridge (CP2102)
 - Ethernet (10/100 Mbps)
- **Power Supply:**
 - Powered via USB or 5V external power using barrel jack

7.1.3 Relevance to the APB Protocol Design

The Nexys 4 board provides an excellent environment for implementing bus protocols like APB due to:

- **Sufficient Logic Resources:** With over 100K logic cells, the Artix-7 FPGA can easily support multiple APB master and slave modules, including RAM, UART, and display peripherals.
- **Peripheral Integration:** The onboard peripherals such as switches, LEDs, and seven-segment displays are used as APB slaves, enabling real-time testing and verification.
- **UART and JTAG Support:** The onboard USB-UART bridge facilitates serial communication testing, and the JTAG interface supports easy programming and debugging.
- **High-Speed Operation:** Artix-7 supports high-frequency operations that enable timing-sensitive bus communication and accurate protocol behavior testing.

7.1.4 Vivado Toolchain Compatibility

The Nexys 4 board is fully compatible with the **Xilinx Vivado Design Suite**, which was used in this project for:

- RTL design and synthesis of the APB protocol
- IP integration and XDC constraint management
- Functional and timing simulation
- Bitstream generation and FPGA configuration

Vivado also provides powerful debugging tools such as the **Integrated Logic Analyzer (ILA)**, which enables real-time signal observation on the FPGA, helping in APB protocol verification.

7.2 FPGA Pin Configuration and Component Mapping

For implementing the APB protocol on the Nexys 4 Artix-7 FPGA board, proper mapping of internal signals to physical FPGA pins is essential. This was achieved through the use of a Xilinx Design Constraints (XDC) file, which assigned the logical ports defined in Verilog to physical I/O locations on the board.

In addition to the XDC configuration, Figure 7.2 illustrates the practical interfacing between the FPGA design and onboard peripherals such as switches, LEDs, buttons, the seven-segment display, and UART ports.

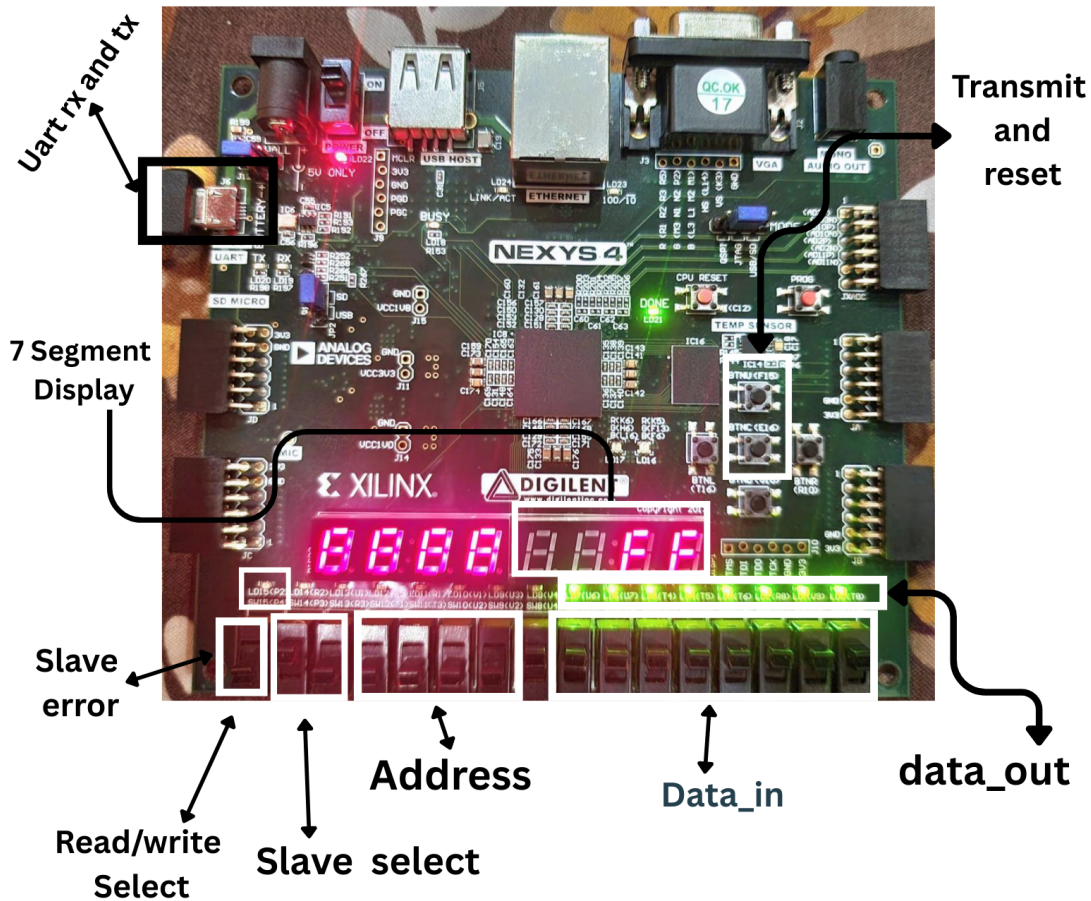


Figure 7.2: Labeled Nexys 4 FPGA board showing APB signal mapping

7.2.1 Clock Signal

The system uses the onboard **100 MHz clock** as the primary timing source. It drives the synchronous logic in the APB master, slave modules, and the UART interface.

7.2.2 User Inputs

The following onboard switches and buttons are used to control and test APB protocol functionalities:

- **Slave Select (SW13-SW14):** These switches select one of the three APB slave devices—RAM, seven-segment display, or UART.
- **Address (SW9–SW12):** Provide the address input for the selected slave module.
- **Read/Write Select (SW15):** Used to choose between read and write operations.

- **Data Input (SW0–SW7):** Input data to be written to the slave device during a write operation.
- **Transmit Button:** Push-button initiates a single APB transaction, ensuring data is sent only once per press.
- **Reset Button(BTNU):** Resets the system during testing or when restarting a session.

7.2.3 Output Indicators

Outputs are displayed using the following onboard resources:

- **Data Output (LEDs LD0–LD7):** Displays data read from the APB slave.
- **Slave Error Indicator (LD15):** Lights up if a transaction addresses a non-existent or incorrectly configured slave.
- **Seven-Segment Display:** Shows numeric output for relevant values like data or address.

7.2.4 UART Interface

The APB UART slave communicates with an external PC through the USB-UART port located at the top left of the board. It uses:

- **UART RX and TX:** Mapped to the board’s USB-UART bridge, enabling communication with terminal software like PuTTY or Tera Term.

7.3 Hardware Setup

To establish communication between the FPGA and PC, connect the RS232 port on the FPGA board to the PC’s serial port (or use a USB-to-RS232 adapter if necessary). On the PC, use Putty to configure the serial communication parameters:

- **COM port selection:** Choose the correct COM port corresponding to the connected RS232 interface.
- **Baud rate:** Set to 115200 bps.
- **Data bits:** 8 data bits.
- **Parity:** No parity.
- **Stop bits:** 1 stop bit.



Figure 7.3: Hardware Setup

- **Flow control:** None.

On the FPGA, configure the UART receiver module to act as the slave. The FPGA will listen for incoming data from the PC and respond accordingly via the RS232 interface.

7.4 Memory Operations

7.4.1 Write Operation

To perform a write operation to the memory, the following signals are used:

- **psel = 2'b01:** Select the memory slave.
- **pwrite = 1:** Enable the write operation.
- **address = 8'h10:** Set the address to where data will be written.
- **data_in = 8'hAB:** Provide the data (0xAB) to be written.

7.4.2 Read Operation

For reading from the memory, the following signals are used:

- **psel = 2'b01:** Select the memory slave.
- **pwrite = 0:** Disable write operation, enable read operation.
- **address = 8'h10:** Set the address from where data will be read.

7.5 7-Segment Display Operations

7.5.1 Write Operation

To write data to the 7-segment display, the following signals are used:

- **psel = 2'b10**: Select the 7-segment display slave.
- **pwrite = 1**: Enable the write operation.
- **data_in = 8'hF2**: Provide the data (0xF2) to be displayed on the 7-segment display.

7.5.2 Read Operation

For reading from the 7-segment display (if applicable), the following signals are used:

- **psel = 2'b10**: Select the 7-segment display slave.
- **pwrite = 0**: Disable write operation, enable read operation.

7.6 UART Operations

7.6.1 Write Operation (Transmit Data)

To transmit data over UART from the FPGA to the PC, the following signals are used:

- **psel = 2'b11**: Select the UART slave.
- **pwrite = 1**: Enable the write operation.
- **data_in = 8'h72**: The data to be transmitted (0x72, which corresponds to the character 'R').
- **send_button = 1**: Trigger the transmission by enabling the send button.

7.6.2 Read Operation (Receive Data)

To receive data from the UART, the following signals are used:

- **psel = 2'b11**: Select the UART slave.
- **pwrite = 0**: Disable write operation, enable read operation.

Chapter 8: Results and Discussion

This chapter presents the results obtained after the successful synthesis, implementation, and analysis of the APB protocol-based system on the FPGA using Xilinx Vivado. The project included the design of an APB master and three APB-compliant slaves: a memory module, a 7-segment display controller, and a UART module.

8.1 RTL Schematic Overview

The RTL schematic generated in Vivado provides a top-level view of the APB system architecture. It illustrates the hierarchy and interconnections between the APB master and its slaves, ensuring proper signal routing as per the APB protocol.

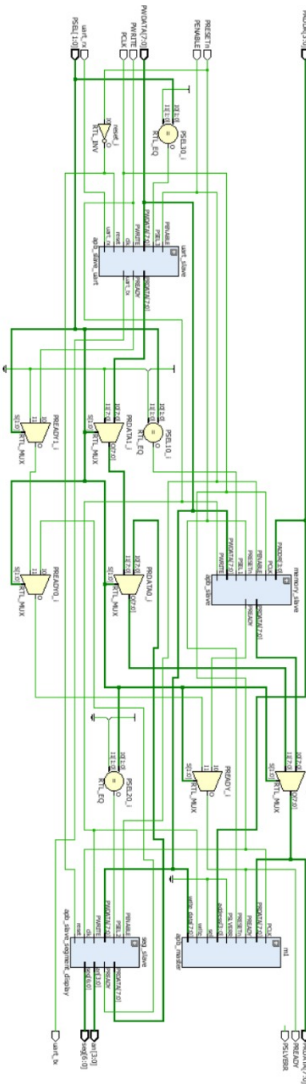


Figure 8.1: RTL schematic of APB Protocol

8.2 Power Analysis Report

Power analysis was conducted post-implementation using Vivado's power estimation tool. The summary of the report is as follows:

- **Total On-Chip Power:** 0.118 W
- **Static Power:** 0.104 W (88%)
- **Dynamic Power:** 0.014 W (12%)
 - Clocks: 0.002 W
 - Signals: 0.001 W
 - Logic: 0.000 W
 - I/O: 0.012 W
- **Junction Temperature:** 25.5°C
- **Thermal Margin:** 59.5°C
- **Confidence Level:** Low (based on constraints/activity estimation)

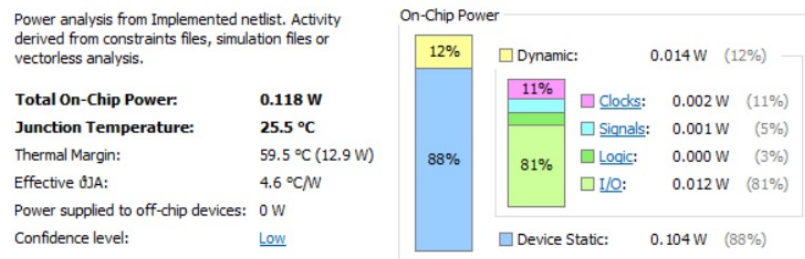


Figure 8.2: Power analysis report of the implemented APB Protocol

8.3 Utilization Report

The utilization report provided by Vivado indicates the resources consumed by the design components on the FPGA:

Table 8.1: FPGA Resource Utilization Summary

Module	LUTs	Registers	Slices	IOBs
apb_system_top	165	246	101	41
memory_slave	51	137	52	0
seg_slave	20	36	19	0
uart_slave	94	73	48	0

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT Flip Flop Pairs (63400)	Bonded IOB (210)	BUFGCTRL (32)
apb_system_top	165	246	16	8	101	165	294	41	1
memory_slave (apb_slave)	51	137	16	8	52	51	146	0	0
seg_slave (apb_slave_seg...)	20	36	0	0	19	20	53	0	0
uart_slave (apb_slave_uart)	94	73	0	0	48	94	110	0	0

Figure 8.3: Vivado utilization report of the APB Protocol

8.4 Discussion

The implemented APB protocol system demonstrates high resource efficiency and low power consumption. Key observations from the results are as follows:

- The **UART slave** consumes the most resources due to the logic required for serial communication.
- The **7-segment display slave** and **memory slave** are lightweight in resource usage.
- The power consumption is low, making the design suitable for embedded and IoT applications.

These results confirm the successful and optimized design and implementation of the APB protocol on an FPGA.

Chapter 9: Applications and Use Cases

The Advanced Peripheral Bus (APB) protocol is widely used in embedded systems, particularly for connecting low-speed peripherals to the system bus. Its simplicity, low power consumption, and ease of integration make it ideal for FPGA-based designs.

9.1 Real-World Applications

1. **SoC and Microcontrollers:** In ARM-based SoCs, APB connects various peripherals, such as UART, SPI, I2C, timers, and GPIOs, offering a lightweight communication mechanism for low-speed devices.
2. **Consumer Electronics:** Devices like smartphones, smartwatches, and TVs utilize APB to interface with internal components that require minimal bandwidth, ensuring efficient communication and power management.
3. **Simple read write operation:** APB allows easy read and write operations with memory-mapped peripherals. It uses signals like PADDR, PWDATA, PWRITE, and PENABLE to perform transactions. This makes it ideal for accessing devices like UART, timers, or small RAM blocks such as DDR interfaces.

9.2 Scope for Further Enhancement

1. **AHB/APB Bridge Implementation:** By integrating an AHB-to-APB bridge, the design could simulate a more complex SoC environment, where high-performance and low-performance buses interact.
2. **Multiple Slave Support:** The design can be extended to support additional APB slaves, such as RAM, displays, or ADCs, allowing for greater system functionality and flexibility.
3. **Low-Power Optimization:** Implementing techniques such as clock gating and power-down modes for idle peripherals could reduce power consumption, further optimizing the system for energy-efficient applications.

Chapter 10: Conclusion

10.1 Summary of the Project

The project titled ”**Design and Implementation of the APB Protocol on FPGA**” aimed to design and implement the Advanced Peripheral Bus (APB) protocol on an FPGA platform, specifically the Artix-7 Nexys 4 board. The APB protocol is widely used in embedded systems for its simplicity and effectiveness in connecting peripherals to a processor. The key objectives of this project were:

- Designing an APB master-slave architecture.
- Implementing various APB slaves, including RAM, a 7-segment display controller, and a UART interface.
- Testing the communication and functionality of these components using FPGA hardware.

10.2 Key Achievements

- **APB Master-Slave Architecture:** A reliable master-slave architecture for APB was successfully developed and tested. The design ensured efficient communication between the master and slaves, with proper timing control and data integrity.
- **Peripheral Integration:** Three different APB slaves (RAM, 7-segment display, and UART) were implemented to showcase the versatility of the APB protocol. The integration with each peripheral was seamless, demonstrating the ability of the APB protocol to support multiple devices.
- **FPGA Implementation:** The APB protocol was fully implemented on the Nexys 4 FPGA using VHDL/Verilog. The FPGA hardware provided a suitable platform for demonstrating the practicality of the APB protocol in real-world applications. All components, including the APB bus controller and peripherals, were synthesized and successfully deployed on the FPGA.
- **Test Scenarios:** Multiple test scenarios were performed to validate the functionality of the APB protocol. These tests ensured that the communication between the master and slave devices was accurate, with no data loss or timing issues.

10.3 Contributions to the Field

This project has made several important contributions to the field of embedded systems and FPGA design:

- **Educational Value:** By designing a complete APB protocol system on an FPGA, this project serves as an educational tool for understanding bus protocols, peripheral integration, and FPGA design.
- **Design Methodology:** The methodology used for implementing and testing the APB protocol can be applied to other bus protocols or FPGA-based systems. The design process, which included functional simulation, hardware synthesis, and real-time testing, can be adapted for similar projects.
- **Peripheral Control:** The integration of a UART, RAM, and a 7-segment display via the APB protocol shows how peripheral devices can be controlled efficiently through a simple communication protocol like APB.

10.4 Limitations and Challenges

While the project achieved its primary objectives, there were several challenges and limitations:

- **Timing Control Issues:** One of the main difficulties encountered during the project was controlling the timing input to the APB and ensuring that data was transmitted only once. Initially, during data transmission, multiple transmissions occurred unexpectedly. This issue was fixed by adding an extra push button, which ensured that the data was transmitted only once.
- **Limited Number of Slaves:** Due to time constraints, only a few peripheral devices were integrated with the APB master. Future work could involve expanding the design to support more complex systems with additional peripherals.
- **Resource Constraints:** The Artix-7 FPGA, while suitable for the project, has limited resources, and this sometimes required careful optimization of the design to fit within the available logic elements and memory.
- **Speed of Communication:** The speed of communication between the master and slaves was constrained by the APB protocol's inherent limitations. Future improvements could focus on optimizing the protocol for higher-speed communication.

10.5 Final Thoughts

The successful implementation of the APB protocol on an FPGA not only demonstrates the practicality of using simple bus protocols for embedded systems but also lays the foundation for future work in FPGA-based peripheral communication systems. The project highlights the importance of hardware-software integration in modern embedded systems and provides valuable insights into FPGA design techniques.

Through careful planning, design, and testing, the project has successfully met its objectives, demonstrating both the power and flexibility of the APB protocol in real-world FPGA applications. The work serves as a solid foundation for further research and development in the area of embedded system design.

References

- [1] Rai, Soumya, and Yadav, Virendra Pratap, Comparison between ARM AMBA Protocols and Verification of APB Protocol Using System Verilog & UVM.
- [2] Neeharika, T, Prasanna, J Lakshmi, Kumar, M Ravi, and Santhosh, Chella, Maximizing efficiency: building and running an AMBA APB protocol with open-source solutions. In *IET Conference Proceedings CP832*, vol. 2023, no. 5, pp. 99–103, 2023.
- [3] Jain, Padmaprabha, and Rao, Satheesh, Design and verification of advanced microcontroller bus architecture-advanced peripheral bus (AMBA-APB) protocol. In *2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*, pp. 462–467. IEEE, 2021.
- [4] Rayasa, Hrishikesh G, Prasad, Hemanth, and Shirur, Yasha Jyothi M, Synthesis of AMBA APB BUS Protocol. In *2025 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, pp. 1–5. IEEE, 2025.
- [5] Yadav, Sirimalla Karun Kalyan, Satyanarayana, P, Krishna, Bavineni Vaibhav, Sai, Kolli Ranjith Kesav, Arun, M, and Krishnan, V Gokula, Design and Implementation of AMBA-APB Protocol for System-on-Chip (SoC) Designs. In *2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, pp. 1–5. IEEE, 2024.
- [6] Bolla, Dileep Reddy, and Palle, Satya Srikanth, Design and Implementation of Advanced Peripheral Bus (APB) Protocol in Systemverilog, 2022.
- [7] Shanthi, G, Sravani, K Girija, Vali, SK Shoukath, Ashwini, Nangunuri, Sainath, Gangaraboina, and Sai, Pyata Gaurav, Design and FPGA implementation of AHB-to-APB bridge. *Microsystem Technologies*, vol. 31, no. 4, pp. 925–934, Springer, 2025.
- [8] HK, Meghana Jain, and Kumar, Punith, Verification of Advanced Peripheral Bus Protocol (APB V2.0), 2021.
- [9] Liu, Lei, Sui, Jinxue, and Zhang, Xia, Design of AXI-APB bus bridge verification platform. In *2023 5th International Academic Exchange Conference on Science and Technology Innovation (IAECST)*, pp. 138–142. IEEE, 2023.

Appendix

Chapter A : APB Protocol Verilog Implementation

1. APB Master Verilog Code

```
1  `timescale 1ns / 1ps
2
3  `timescale 1ns / 1ps
4
5  module apb_master #(
6      parameter N = 8,
7      parameter DEPTH = 256
8  )(
9      input PCLK,
10     input PRESETn,
11     input PREADY,
12     input [N-1:0] PRDATA,
13     input write,
14     input sel,
15     input [$clog2(DEPTH)-1:0] address,
16     input [N-1:0] write_data,
17     input PSLVERR,
18
19     output reg PSEL1,
20     output reg PENABLE,
21     output reg [$clog2(DEPTH)-1:0] PADDR,
22     output reg [N-1:0] PWDATA,
23     output reg PWRITE
24 );
25     reg [1:0] state, next_state;
26
27     parameter IDLE    = 2'b00,
28                 SETUP  = 2'b01,
29                 ACCESS = 2'b10;
30
31     always @(posedge PCLK or negedge PRESETn) begin
32         if (!PRESETn) begin
33             state    <= IDLE;
34             PSEL1    <= 0;
35             PADDR    <= 0;
36             PWDATA   <= 0;
37             PWRITE   <= 0;
```

```

38         PENABLE <= 0;
39     end else begin
40         state <= next_state;
41     end
42 end
43
44 always @(PREADY or sel or write or state) begin
45     case (state)
46     IDLE: begin
47         PENABLE = 0;
48         PSEL1    = sel;
49         if (PSEL1)
50             next_state = SETUP;
51         else
52             next_state = IDLE;
53     end
54
55     SETUP: begin
56         PENABLE = 0;
57         PADDR    = address;
58         PWRITE   = write;
59         PSEL1    = sel;
60         if (PWRITE)
61             PWDATA = write_data;
62         next_state = ACCESS;
63     end
64
65     ACCESS: begin
66         PENABLE = 1;
67         if (PREADY) begin
68             if (!PSEL1)
69                 next_state = IDLE;
70             else
71                 next_state = SETUP;
72         end else begin
73             next_state = ACCESS;
74         end
75     end
76
77     default: next_state = IDLE;
78 endcase
79 end
80 endmodule

```

Listing 1: APB Master Verilog Module

2. APB Slave Verilog Code

```
1
2 `timescale 1ns / 1ps
3
4 module apb_slave #(
5     parameter N = 8,
6     parameter DEPTH = 256,
7     parameter valid_address = 170
8 ) (
9     input [$clog2(DEPTH)-1:0] PADDR,
10    input [N-1:0] PWDATA,
11    input PCLK,
12    input PRESETn,
13    input PSEL1,
14    input PENABLE,
15    input PWRITE,
16    output reg [N-1:0] PRDATA,
17    output reg PREADY,
18    output reg PSLVERR
19 );
20
21    reg [7:0] mem[0:DEPTH - 1];
22
23    // Address range is from 0 to valid_address (e.g., 0 to 170).
24    // If out of range, an error is generated.
25
26    always @(posedge PCLK or negedge PRESETn) begin
27        if (!PRESETn) begin
28            PRDATA    <= 0;
29            PREADY    <= 0;
30            PSLVERR   <= 0;
31        end
32    end
33
34    always @(*) begin
35        if (PSEL1 && PWRITE && PENABLE) begin // Write operation
36            if (PADDR >= 0 && PADDR <= valid_address) begin
37                PSLVERR    = 0;
38                PREADY     = 1;
39                mem[PADDR] = PWDATA;
40            end else begin
41                PSLVERR = 1;
42                PREADY  = 1;
43            end
44        end
45    end
```

```

44     end else if (PSEL1 && !PWRITE && PENABLE) begin // Read
        operation
45         if (PADDR >= 0 && PADDR <= valid_address) begin
46             PSLVERR = 0;
47             PREADY  = 1;
48             PRDATA   = mem[PADDR];
49         end else begin
50             PSLVERR = 1;
51             PRDATA   = 8'hxx;
52             PREADY  = 1;
53         end
54     end else begin
55         PREADY  = 0;
56         PSLVERR = 0;
57     end
58 end
59
60 endmodule

```

Listing 2: APB Slave Verilog Module

3. APB Top-Level Integration Module

```
1  `timescale 1ns / 1ps
2
3  `include "master.v"
4  `include "slave_1.v"
5
6  module apb #(
7      parameter N = 8,
8      parameter DEPTH = 256
9  )(
10     input PCLK,
11     input PRESETn,
12     input PWRITE,
13     input PSEL1,
14     input [$clog2(DEPTH)-1:0] PADDR,
15     input [N-1:0] PWDATA,
16
17     output reg [N-1:0] PRDATA,
18     output reg PREADY,
19     output reg PSLVERR
20 );
21
22 // Internal connections from master to slave
23 wire [N-1:0] wdata;
24 wire [$clog2(DEPTH)-1:0] addr;
25 wire write, enable, sel;
26
27 // Master Instance
28 apb_master m1 (
29     .PCLK(PCLK),
30     .PRESETn(PRESETn),
31     .write(PWRITE),
32     .sel(PSEL1),
33     .address(PADDR),
34     .write_data(PWDATA),
35     .PRDATA(PRDATA),
36     .PSLVERR(PSLVERR),
37     .PREADY(PREADY),
38     .PWRITE(write),
39     .PSEL1(sel),
40     .PADDR(addr),
41     .PWDATA(wdata),
42     .PENABLE(enable)
43 );
```

```

44
45 // Slave Instance
46 apb_slave s1 (
47     .PCLK(PCLK),
48     .PRESETn(PRESETn),
49     .PSEL1(sel),
50     .PENABLE(enable),
51     .PWRITE(write),
52     .PADDR(addr),
53     .PWDATA(wdata),
54     .PRDATA(PRDATA),
55     .PSLVERR(PSLVERR),
56     .PREADY(PREADY)
57 );
58
59 endmodule

```

Listing 3: Top-Level APB Module Integrating Master and Slave

4. APB Testbench

```
1  `timescale 1ns / 1ps
2
3  module apb_tb #(parameter N = 8,
4                      parameter DEPTH = 256);
5
6      reg PCLK, PRESETn, PWRITE, PSEL1;
7      reg [$clog2(DEPTH)-1:0] PADDR;
8      reg [N-1:0] PWDATA;
9
10     wire [N-1:0] PRDATA;
11     wire PSLVERR, PREADY;
12
13     apb dut (
14         .PCLK(PCLK),
15         .PRESETn(PRESETn),
16         .PWRITE(PWRITE),
17         .PSEL1(PSEL1),
18         .PADDR(PADDR),
19         .PWDATA(PWDATA),
20         .PRDATA(PRDATA),
21         .PSLVERR(PSLVERR),
22         .PREADY(PREADY)
23     );
24
25     initial begin
26         $monitor("RESET=%0d, PWRITE=%0d, PSEL1=%0d, PREADY=%0d, PADDR=%0d,
27                 PWDATA=%0d, PRDATA=%0d, PSLVERR=%0d, PENABLE=%0d",
28                 PRESETn, PWRITE, PSEL1, PREADY, PADDR, PWDATA, PRDATA,
29                 PSLVERR, dut.enable);
30     end
31
32     task write(input [7:0] addr, data);
33         PSEL1 = 1;
34         @(posedge PCLK);
35         PWRITE = 1;
36         PADDR = addr;
37         PWDATA = data;
38         @(posedge PCLK);
39     endtask
40
41     task read(input [7:0] addr);
42         PSEL1 = 1;
43         @(posedge PCLK);
```

```

42     PWRITE = 0;
43     PADDR = addr;
44     @(posedge PCLK);
45 endtask
46
47 task reset();
48     PRESETn = 0;
49     PSEL1 = 0;
50     #22 PRESETn = 1;
51 endtask
52
53 initial begin
54     PCLK = 1;
55     reset;
56
57     // Simple write & read
58     if ($test$plusargs("sanity_test")) begin
59         $display("Simple read-write operation is going on");
60         write(8'h01, 8'h11);
61         read(8'h01);
62         #12 reset;
63     end
64
65     // Multiple write & read
66     if ($test$plusargs("multiple_write_read_test")) begin
67         $display("Multiple read-write operation");
68
69         write(8'h02, 8'h22);
70         write(8'h03, 8'h33);
71         write(8'h04, 8'h44);
72         write(8'h05, 8'h55);
73         write(8'hAA, 8'h89);
74
75         read(8'h02);
76         read(8'h03);
77         read(8'h04);
78         read(8'h05);
79         read(8'hAA);
80
81         #72 reset;
82     end
83
84     // Back-to-back test
85     if ($test$plusargs("back2back")) begin
86         $display("Back-to-back read-write operation");

```

```

87
88     write(8'h02, 8'h22);
89     read(8'h02);
90     write(8'h03, 8'h33);
91     read(8'h03);
92     write(8'h04, 8'h44);
93     read(8'h04);
94     write(8'h05, 8'h55);
95     read(8'h05);
96     write(8'hAA, 8'h89);
97     read(8'hAA);
98
99     #72 reset;
100 end
101
102 // Asynchronous reset test
103 if ($test$plusargs("multiple_write_async_reset_read_test")) begin
104     $display("Multiple write with async reset read test");
105
106     write(8'h02, 8'h22);
107     write(8'h03, 8'h33);
108     write(8'h04, 8'h44);
109     write(8'h05, 8'h55);
110
111     #43 reset;
112
113     read(8'h02);
114     read(8'h03);
115     read(8'h04);
116     read(8'h05);
117     #12 reset;
118 end
119
120 // Unauthorized location access
121 if ($test$plusargs("unauthorized_location_error_test")) begin
122     $display("Unauthorized location error test");
123     write(8'hBB, 8'h99);
124     read(8'hBB);
125     #12 reset;
126 end
127 end
128
129 always #5 PCLK = ~PCLK;
130
131 initial begin

```

```
132     $dumpfile("apb_tb.vcd");  
133     $dumpvars;  
134     #500 $finish;  
135 end  
136  
137 endmodule
```

Listing 4: Testbench for APB Master-Slave Design